Networks Assignment - 03 Lab Report
Group 07

NS-3 (Network Simulator)

NS-3 (Network Simulator 3) is a discrete-event network simulation framework primarily used for research and teaching in computer networking. It provides a realistic and flexible environment to model wired, wireless, and hybrid networks at various layers of the protocol stack — from the physical layer (PHY) up to the application layer.

Unlike emulators that require real network interfaces, NS-3 simulates packet exchanges, protocol behavior, and timing entirely in software using an event scheduler. This makes it ideal for studying protocol performance, routing dynamics, congestion control, and reliability mechanisms without real hardware.

1. NS-3 Architecture Overview

NS-3 is written in C++ (with optional Python bindings) and provides a modular, object-oriented design. Its architecture consists of the following major components:

Layer	Component	Description
Application Layer	*OnOffApplication*, *UdpEchoClient/Server*, or custom apps	Generates and receives simulated traffic (TCP, UDP, HTTP, FTP, etc.)
Transport Layer	*TCP*, *UDP*, *Custom RDT Protocols (ABP, GBN)*	Implements transport-level logic and flow control
Network Layer	*IPv4*, *Routing Protocols (DV, LS)*	Handles packet forwarding and routing decisions
Data Link / MAC Layer	*PointToPointNetDevice*, *Wi-Fi*, *CSMA*, etc.	Models link-layer behavior and queueing
Physical Layer	*Channel*, *ErrorModel*	Represents bandwidth, delay, and signal propagation
Utilities	*Flow Monitor*, *Packet Tagging*, *Tracing*	Used for collecting simulation metrics and statistics

2. Event-Driven Simulation

In NS-3, every operation (packet transmission, reception, timeout, ACK, etc.) is represented as a scheduled event in simulated time. The simulation clock advances by processing these events in chronological order.

Example:
If a packet is sent at t = 0.01 s and the link delay is 10 ms, the reception event is scheduled at t = 0.02 s.

This deterministic timing allows repeatable experiments, independent of real CPU timing or hardware performance.

3. Summary

NS-3 enables detailed, reproducible experiments for protocol analysis, performance measurement, and design validation.

Through modules such as PointToPoint, Flow Monitor, and ErrorModel, it provides a complete software environment to emulate realistic networking conditions like packet loss, delay, and retransmissions — making it ideal for this assignment's problems on Reliable Data Transfer, TCP Congestion Control, HTTP Proxy performance, and Distance Vector routing.

Problem 1: Reliable Data Transfer using NS-3

1. Objective

The primary objective of this task is to understand and implement **Reliable Data Transfer (RDT)** protocols using the NS-3 network simulator. The task involves:

- Implementing two fundamental reliability mechanisms:
  (a) Alternating Bit Protocol (ABP / Stop-and-Wait)
  (b) Go-Back-N (GBN) Protocol
- Simulating packet loss and corruption over an unreliable link.
- Measuring and analyzing throughput, retransmissions, and delay.

2. Background Theory

- Reliable data transfer (RDT) is a fundamental requirement of computer networking, ensuring that information sent across unreliable communication channels reaches its destination accurately, completely, and in the correct order.
- In practice, underlying physical and link layers are prone to packet loss, bit corruption, duplication, and reordering.
-

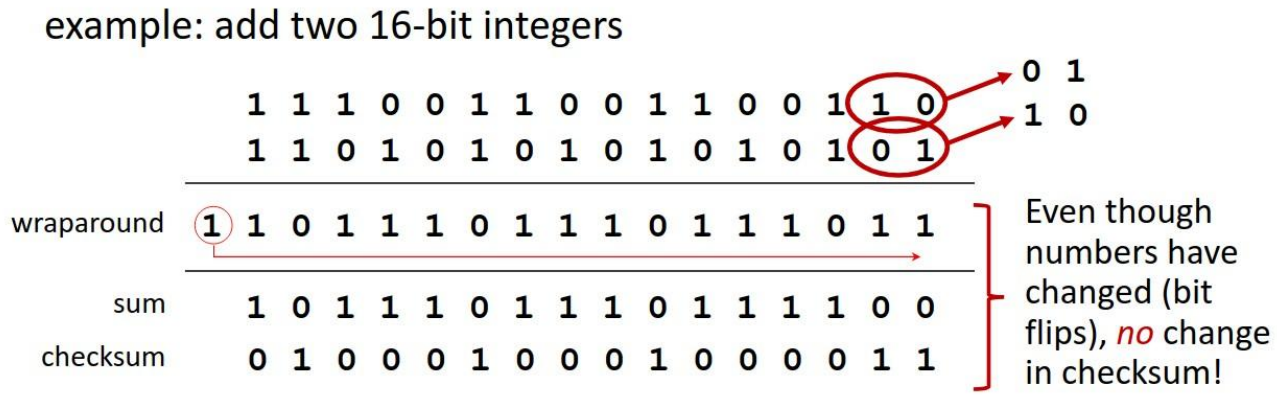
To overcome these problems, reliable transport mechanisms are built on top of such unreliable channels using **error detection, acknowledgement, timeouts, and retransmission strategies.**

2.1 Automatic Repeat reQuest (ARQ) and Reliability Principles

ARQ (Automatic Repeat reQuest) protocols form the theoretical backbone of reliable data transfer. They combine **error detection** and **feedback control** to request retransmission of corrupted or lost packets. Core concepts include:

Error Detection:

- Each frame carries a **checksum** or **Cyclic Redundancy Check (CRC)** allowing the receiver to detect bit errors.
- Corrupted frames are discarded and re-requested.



Acknowledgements (ACK) and Negative Acknowledgements (NAK):

- The receiver sends an **ACK** for correctly received data or a **NAK** when an error is detected.
- If the sender fails to receive an ACK before the timer expires, it retransmits.

Sequence Numbers:

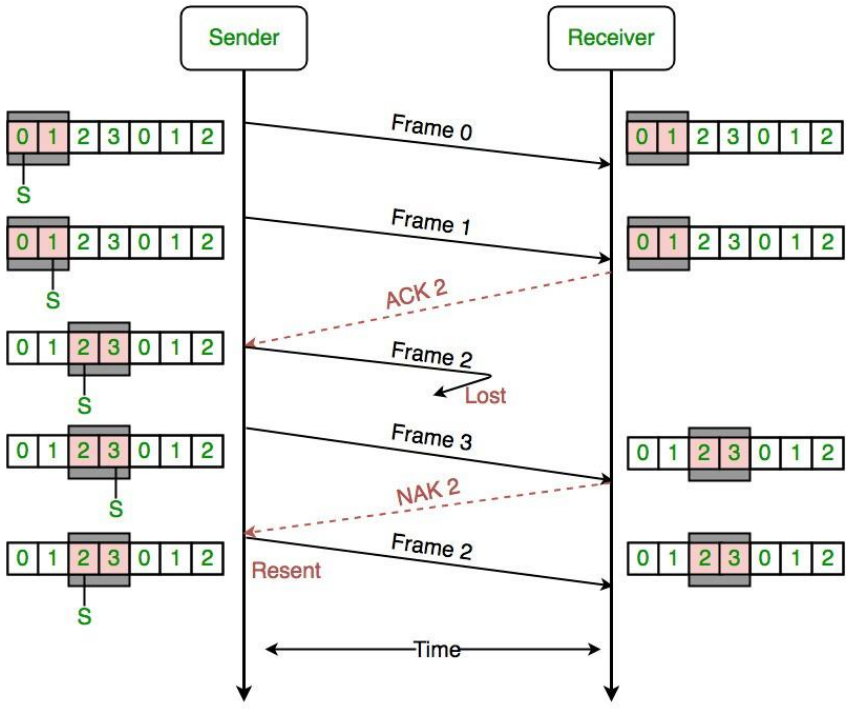
- Every data frame is labeled with a unique sequence number to distinguish new transmissions from retransmissions.
- The receiver uses it to maintain in-order delivery and avoid duplicates.

Timeout and Retransmission:

- The sender starts a timer for each outstanding packet.
- Expiration of the timer triggers retransmission, assuming the packet or its ACK was lost.

Sliding Window Mechanism:

- Instead of sending one packet at a time, a window of packets can be “in flight” without waiting for individual acknowledgements.
- Improves link utilization, especially when **propagation delay** ≫ **transmission time**.



2.2 Alternating Bit Protocol (ABP / Stop-and-Wait ARQ)

The **Alternating Bit Protocol (ABP)**—also called **Stop-and-Wait**—is the simplest ARQ mechanism and serves as the conceptual foundation of all reliable data transfer protocols.

Working Principle:

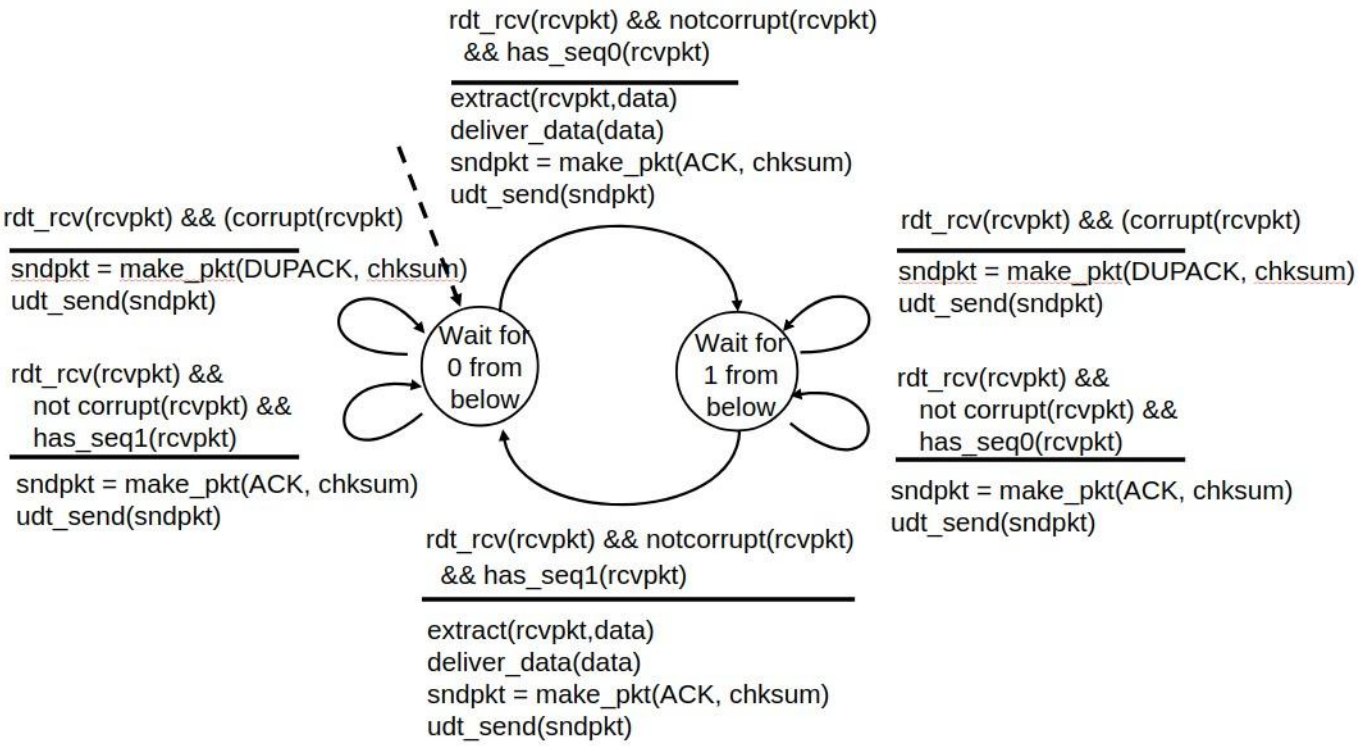
1. The sender transmits a single packet with sequence number 0 or 1.
2. The receiver validates the packet using its checksum.
   a. If correct -> sends ACK with the same sequence number.
   b. If corrupted -> sends dup ACK(sender times out and retransmits).
3. Upon receiving the correct ACK, the sender toggles the sequence bit (from 0 to 1 or vice-versa) and sends the next packet.

FSM Model:

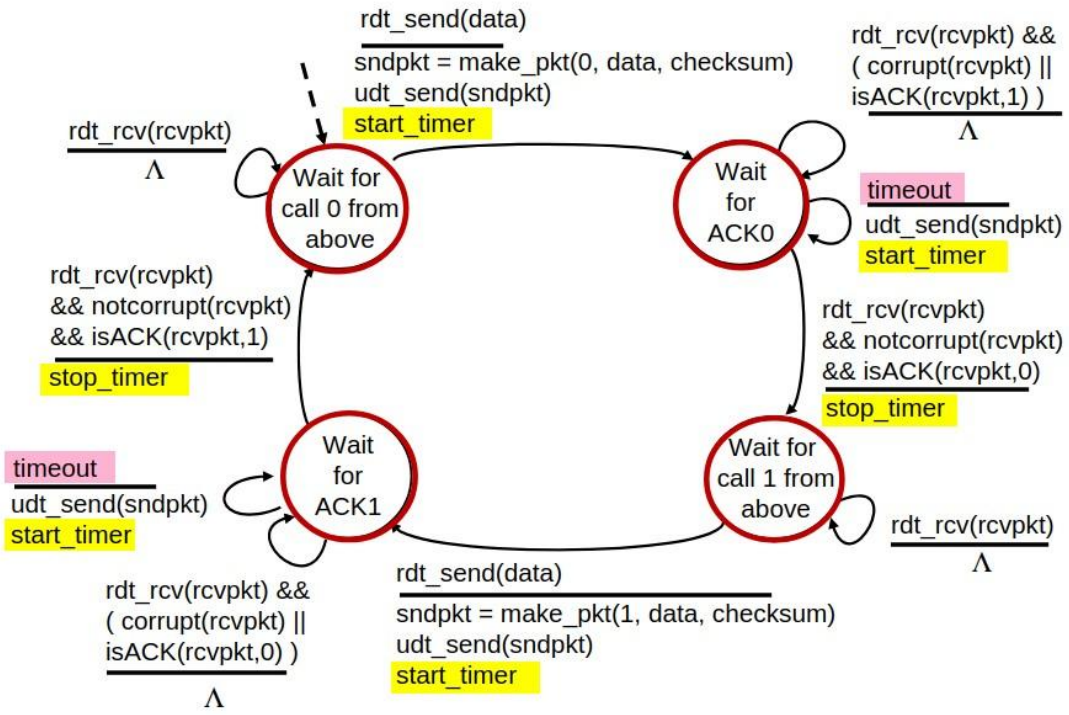
The receiver can be represented as **finite-state machines** with two states (expecting 0 or 1) whereas the sender can be represented as **finite-state machines** with four states . This simple structure ensures:

- No duplicates delivered.
- No data lost (because of retransmission).
- Packets are always delivered in order.

RECEIVER


rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)
&& has_seq0(rcvpkt)
extract(rcvpkt,data)
deliver_data(data)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)
sndpkt = make_pkt(DUPACK, chksum)
udt_send(sndpkt)
rdt_rcv(rcvpkt) &&
not corrupt(rcvpkt) &&
has_seq1(rcvpkt)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)
Wait for
0 from
below
Wait for
1 from
below
rdt_rcv(rcvpkt) && (corrupt(rcvpkt)
sndpkt = make_pkt(DUPACK, chksum)
udt_send(sndpkt)
rdt_rcv(rcvpkt) &&
not corrupt(rcvpkt) &&
has_seq0(rcvpkt)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)
rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)
&& has_seq1(rcvpkt)
extract(rcvpkt,data)
deliver_data(data)
sndpkt = make_pkt(ACK, chksum)
udt_send(sndpkt)

SENDER

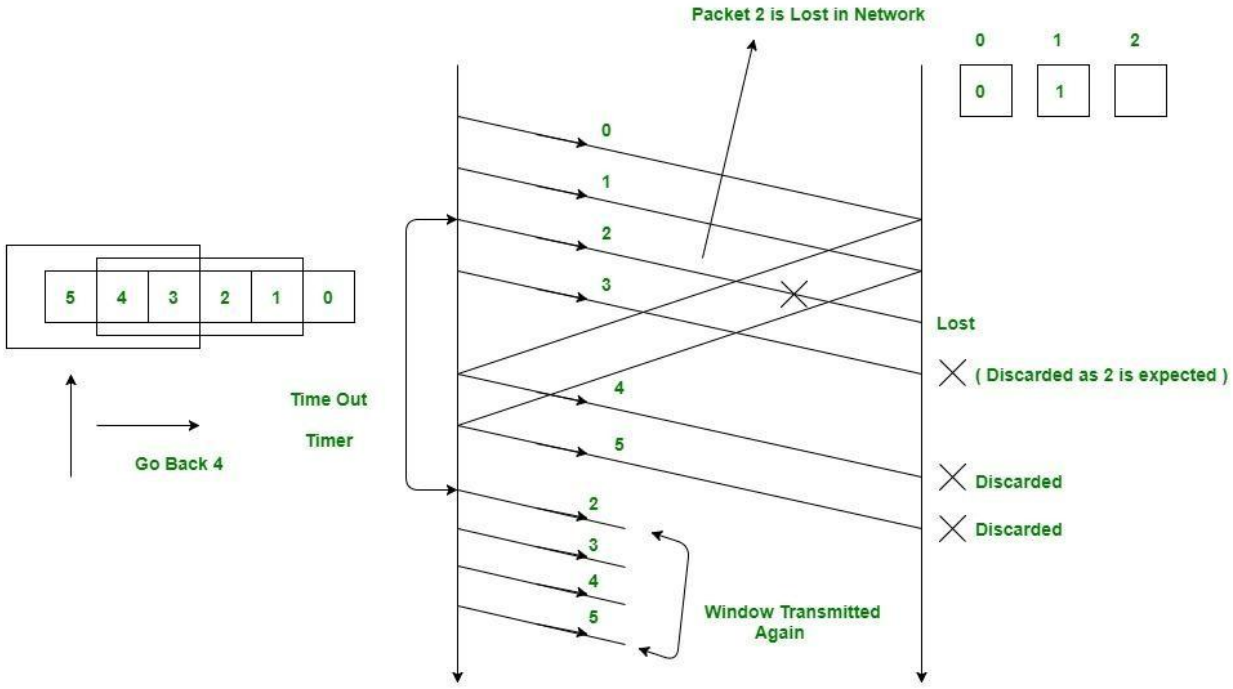


2.3 Go-Back-N (GBN) Protocol

The Go-Back-N (GBN) protocol generalizes ABP using a sliding window of size N that allows multiple unacknowledged packets to be transmitted before waiting for feedback.

Mechanism:

1. The sender maintains two pointers:
   - ○ base → sequence number of the oldest unacknowledged packet. ○ nextseqnum → next packet to send.
2. The window can hold up to N outstanding packets (base ≤ seq < base+N).
3. Receiver uses cumulative acknowledgements: ACK(k) confirms correct reception of all packets ≤ k.
4. If a packet or ACK is lost, sender waits for timeout and then retransmits all packets from the missing one onward (hence “go back N”).
5. Receiver discards any out-of-order packets.



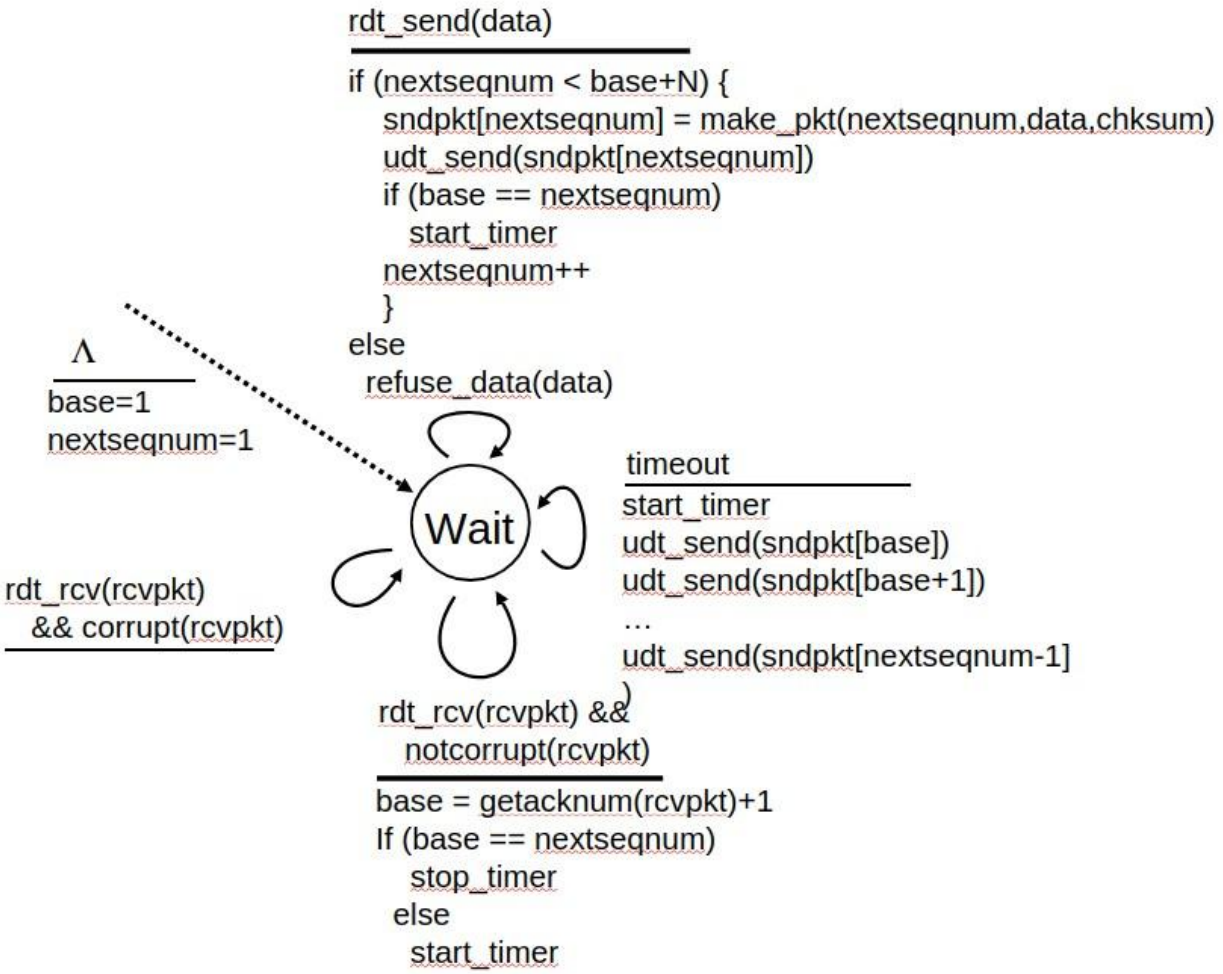
Window and Sequence Number Relation

If N is the window size, the sequence number space must be ≥ 2^N - 1 to avoid ambiguity during wrap-around.

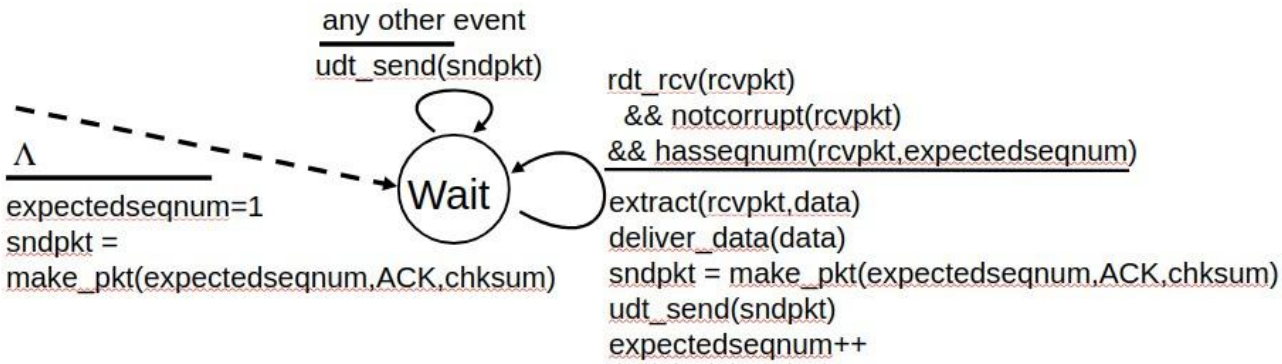
FSM Model:

The receiver can be represented as **finite-state machines** with two states (expecting 0 or 1) whereas the sender can be represented as **finite-state machines** with four states

SENDER



RECEIVER



Performance Characteristics

- Efficiency $\eta \approx 1 / (1 + 2a)$, where a = propagation delay / transmission time.

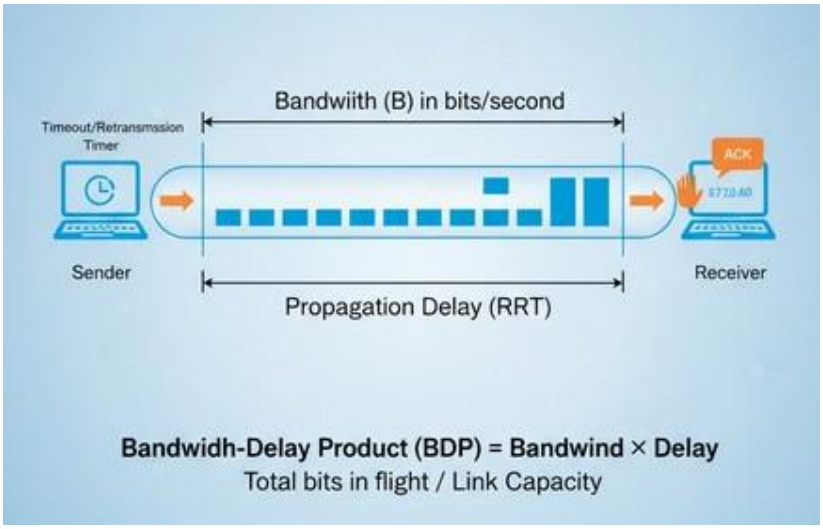
- Works well for short-delay, low-bandwidth links but becomes inefficient when the channel's bandwidth-delay product is large, because the sender remains idle during ACK waiting time.

Advantages & Drawbacks

Aspect	Advantage	Limitation
Throughput	High utilization when errors are rare	Wasted retransmissions if one packet lost
Receiver	Simpler buffer logic	Cannot accept out-of-order packets
Complexity	Moderate	Needs accurate timer management

2.4 Timing Relationships and Channel Utilization

1. Transmission Time (T): time to send a packet = packet size / link bandwidth.
2. Propagation Delay (T): signal travel time between nodes.
3. Round-Trip Time (RTT): ≈ 2T + ACK time + processing delay.
4. Bandwidth–Delay Product (BDP): = Bandwidth × Delay; represents how many bits "fit" in transit.
   - Efficient utilization requires a window ≥ BDP / packet size.



- In Stop-and-Wait, only one packet occupies the link, wasting capacity when BDP ≫ packet size.
- GBN mitigates this by maintaining several packets in transit, approximating full utilization when N ≥ BDP / packet size.

2.5 Error Models and Loss Simulation in NS-3

In NS-3, packet loss and corruption are modeled using RateErrorModel, allowing adjustable probabilities for:

- Bit errors → checksum failures.
- Packet drops → loss events.
- ACK loss → forced retransmissions.

Such artificial impairments reproduce real network unreliability and are essential for evaluating how ABP and GBN handle errors under controlled conditions.

2.6 Protocol Correctness and Formal Properties

Protocols like ABP and GBN are designed to satisfy:

Safety: no corrupted or duplicated data delivered to the application.

Liveness: every packet is eventually delivered (given bounded loss and retries).

In-order Delivery: receiver passes data upward strictly in sequence number order.

Finite-state analysis and model checking have verified these properties formally.

2.7 Comparative Analysis of ABP and GBN

Feature	Alternating Bit Protocol	Go-Back-N Protocol
Window Size	1	N (>1)
Acknowledgement Type	Individual ACK	Cumulative ACK
Sender Concurrency	Single packet	Multiple packets
Receiver Buffering	Not needed	Not required for basic GBN
Retransmission Strategy	Retransmit same packet	Retransmit from first un-ACKed packet
Utilization	Low	High
Implementation Complexity	Simple	Moderate
Suitability	Low-delay networks	High-bandwidth or long-delay links

2.8 Relevance to Modern Protocols

Modern reliable transport mechanisms such as TCP (Transmission Control Protocol) evolve from these ARQ concepts:

- TCP's cumulative ACKs and sliding window (flow control) are direct generalizations of GBN.
- Fast retransmit and selective acknowledgement (SACK) extend the idea toward Selective Repeat. Understanding ABP and GBN thus provides a conceptual foundation for analyzing and tuning TCP behavior in real-world networks. **ABP**

ABP Concept	Description	TCP Feature
Sequence Number (1-bit: 0 or 1)	Used to detect duplicate data packets.	**Sequence Number (32-bit)**
Acknowledgement (ACK)	Receiver sends an ACK for the last received packet to signal the next expected sequence number.	**Acknowledgement Number**
Timeout/ Retransmission	If a data packet's ACK is not received within a set time, the packet is retransmitted.	**Retransmission Timer**

GBN

GBN Concept	Description	TCP Feature
Sliding Window	The sender can transmit multiple packets (window size N) before waiting for an ACK.	**Sending Window (Congestion/Flow Control)**
Cumulative ACK	The receiver sends an ACK that specifies the sequence number of the *next* expected packet, implicitly acknowledging all preceding packets.	**Acknowledgement Number**
Go-Back-N Retransmission	Upon a timeout or error, the sender retransmits the lost packet **and all subsequent packets** that were already sent.	**Cumulative Acknowledgement + Basic Retransmission**

2.9 Summary

Reliable Data Transfer protocols bridge the gap between unreliable physical media and dependable communication demanded by applications.

● ABP demonstrates correctness and simplicity at the expense of efficiency. ● GBN introduces pipelining to increase utilization while maintaining reliability through retransmission on error.

Both were successfully modeled in NS-3 to analyze retransmission counts, throughput, and delay under varying loss probabilities, providing practical insight into the principles that govern modern network reliability.

3. IMPLEMENTATION DETAILS

Simulation Setup in NS-3

- **Topology:** Two nodes connected by a point-to-point channel.

- **Link parameters:** Bandwidth = 1 Mbps, Delay = 10 ms.
- **Error model:** Use RateErrorModel with configurable packet loss and corruption probabilities.
- **Simulation time:** 20 seconds.

Task A: Alternating Bit Protocol (ABP / Stop-and-Wait)

File: abp.cc

Key Features:

1. Custom Header (RDTHeader):
    - Fields: sequence_number, ack_number, check_sum.
    - Enables packet identification and ACK verification.
    - Supports serialization/deserialization for UDP packet exchange.
2. Sender Logic:
    - Send one packet at a time.
    - Waits for an ACK before sending the next.
    - Uses a timer (timeout_interval = 0.2 s) for retransmissions in case ACK is lost.
    - Updates sequence number alternately between 0 and 1.
3. Receiver Logic:
    - Checks received packet sequence number against expected value.
    - Send ACK if correct, or duplicate ACK for out-of-order packets. ○ Maintains cumulative throughput metrics.
4. Metrics Collected:
    - Retransmissions: Number of packets resent due to timeout.
    - Throughput: Calculated as cumulative received bits per second.
    - Output files: retransmissions-task1a.txt and throughput-task1a.txt.
5. Simulation Setup:
    - Two nodes, point-to-point link (1 Mbps, 10 ms delay).
    - RateErrorModel applied with 20% packet loss to simulate an unreliable network.
    - UDP sockets for data transfer and ACKs.

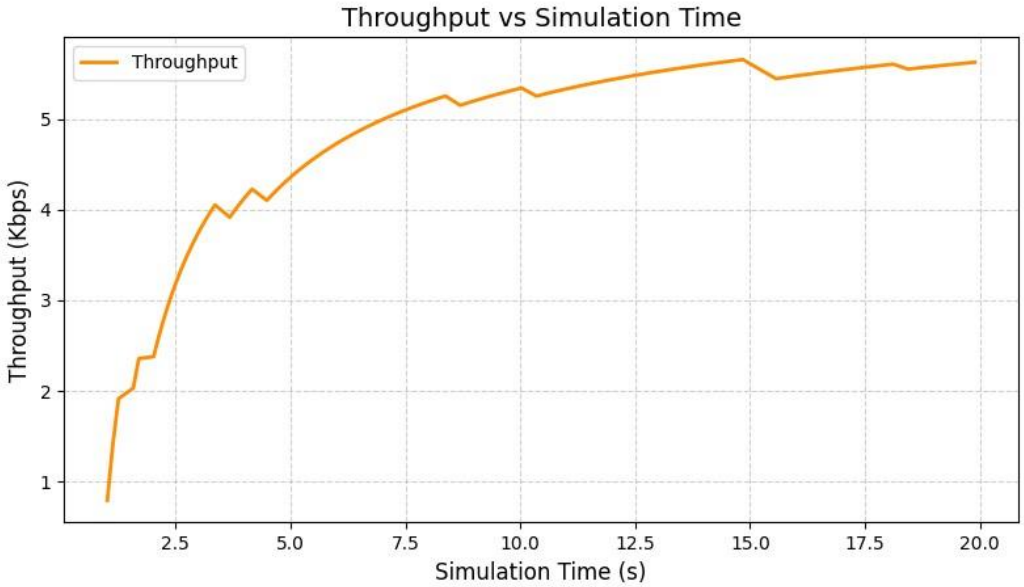
Characteristics of ABP:

- Simple and reliable.
- Only one packet is in transit, so link utilization is low.

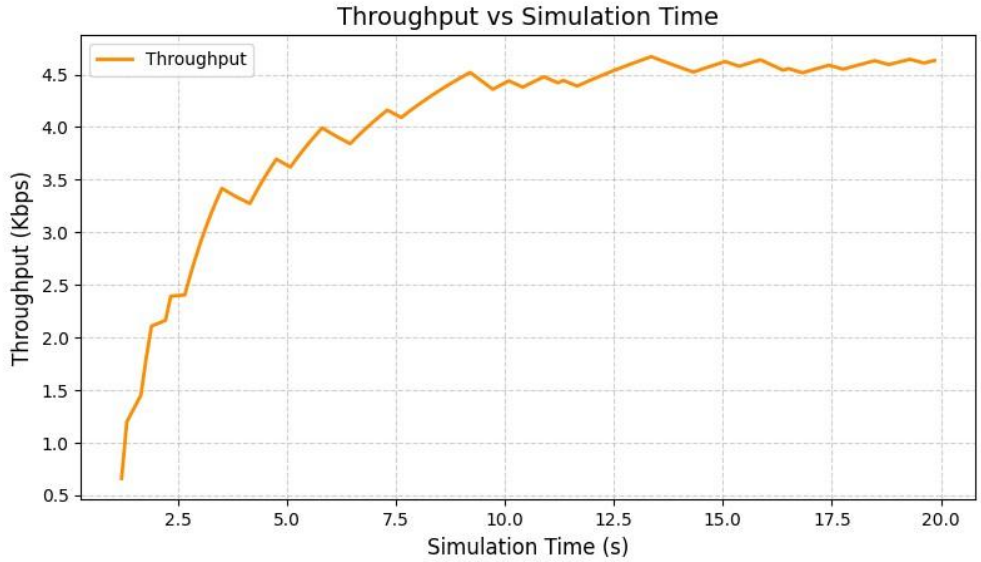
- Retransmissions occur whenever packets or ACKs are lost.

SIMULATION RESULTS

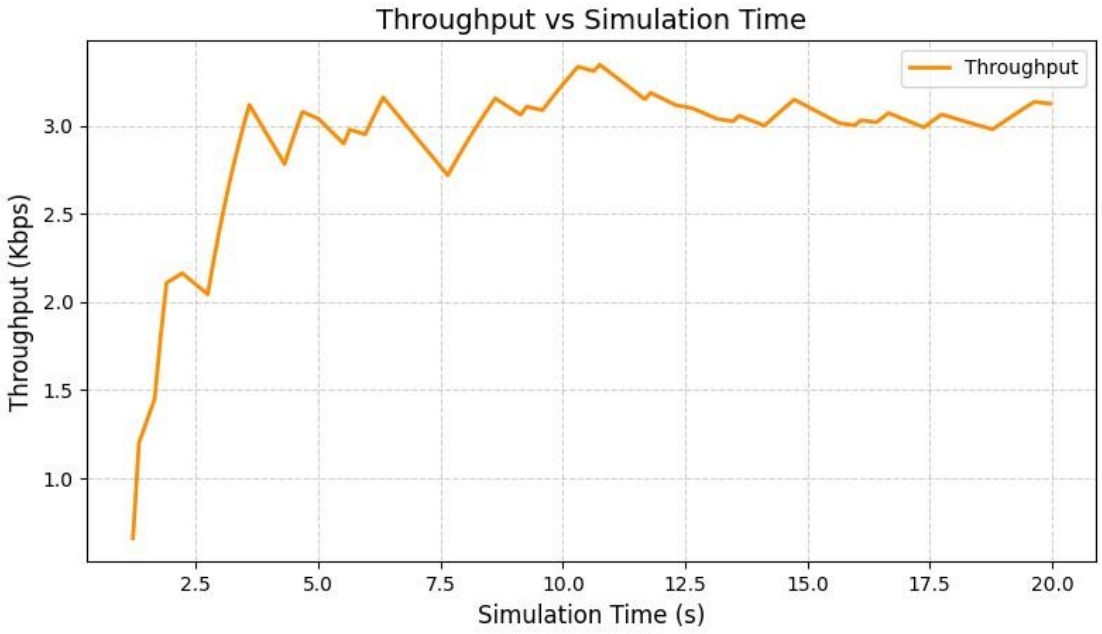
1. Throughput Vs Simulation Time with 5% error rate



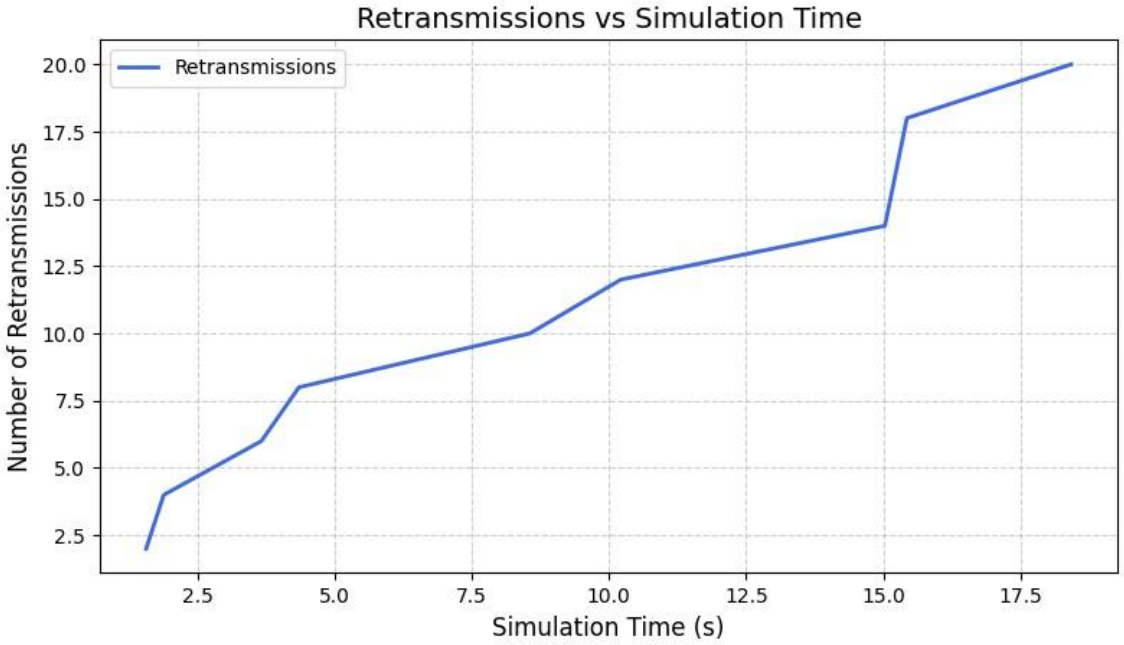
2. Throughput Vs Simulation Time with 10% error rate



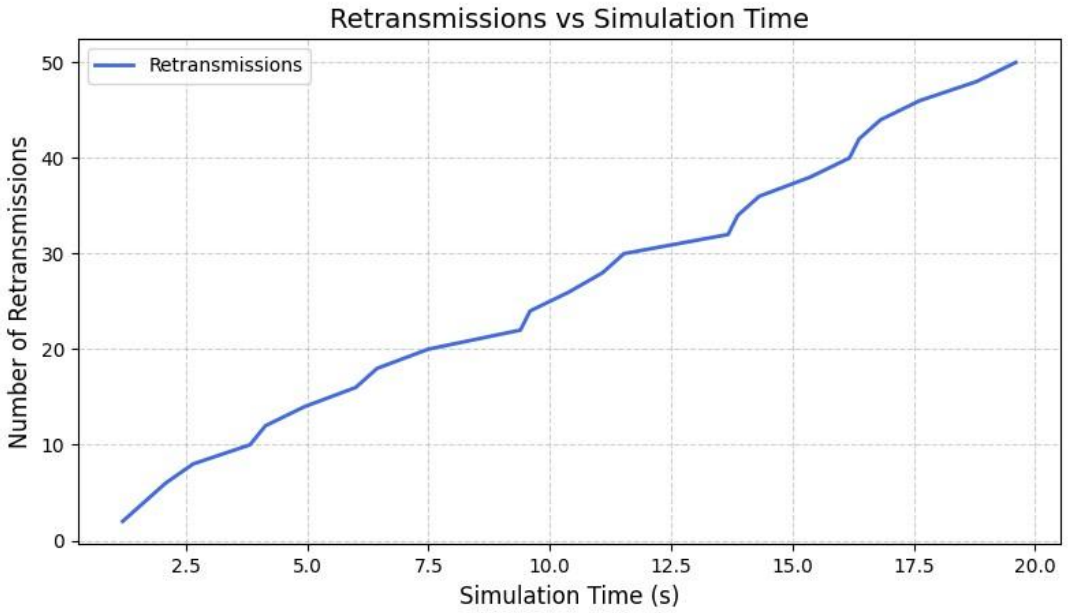
3. Throughput Vs Simulation Time with 20% error rate



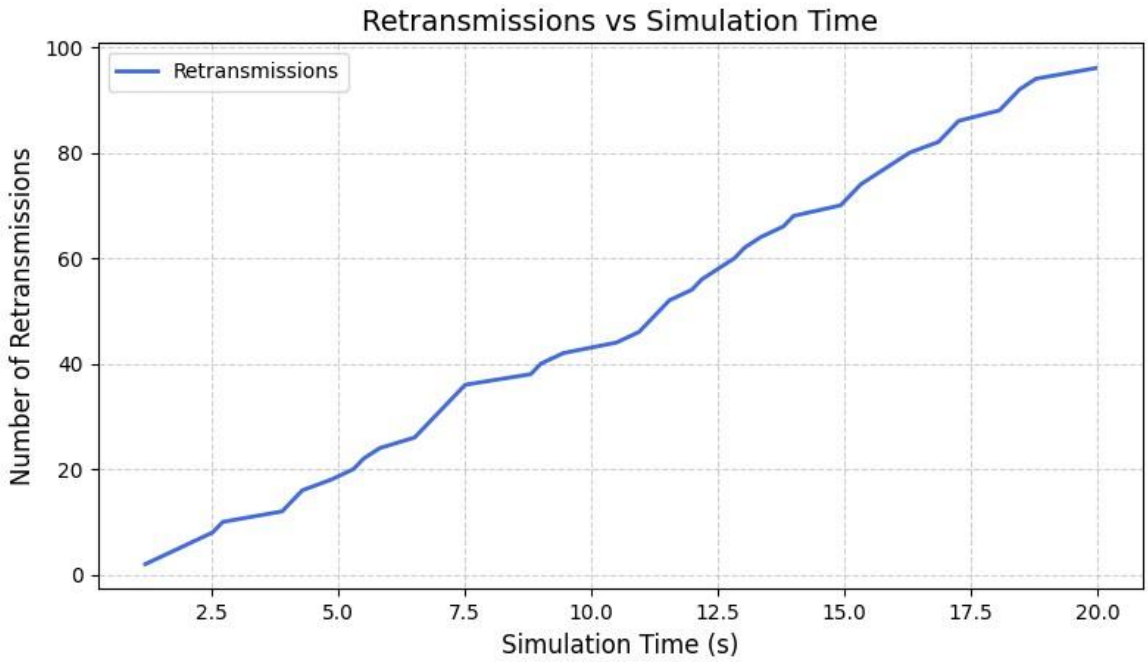
4. Retransmission Vs Simulation time with with 5% error rate



5. Retransmission Vs Simulation time with with 10% error rate



6. Retransmission Vs Simulation time with with 20% error rate



File: gbn.cc

Key Features:

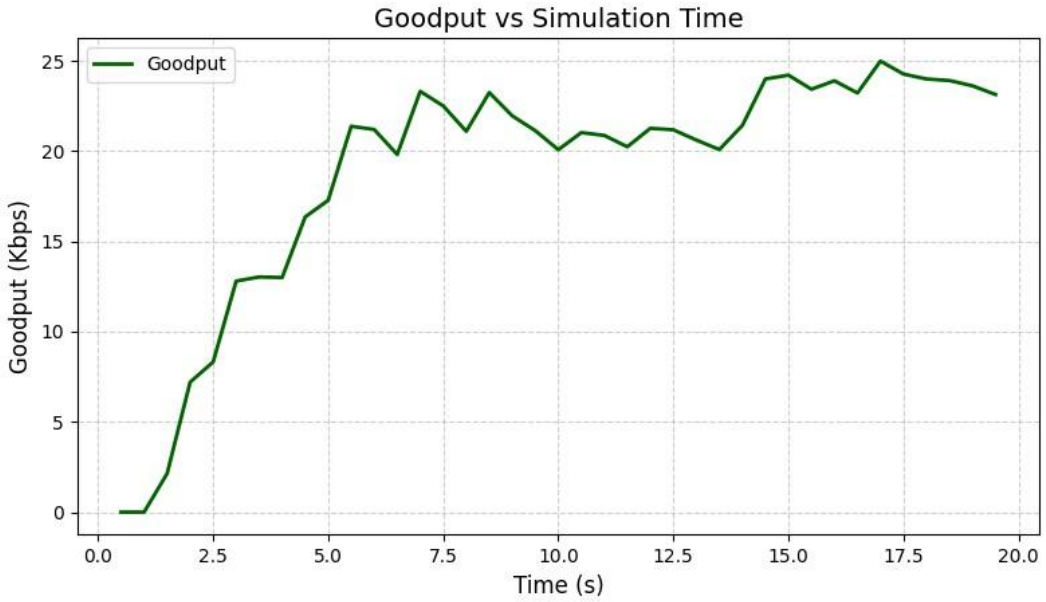
1. Custom Header (RDTHeader):
    - Fields: sequence_number, ack_number, check_sum.
    - Used for identifying packets and sending cumulative ACKs.
2. Sender Logic:
    - Maintains a sliding window of size N = 4.
    - base → sequence number of the oldest unacknowledged packet.
    - next_seq_number → sequence number of the next packet to send.
    - Sends packets continuously while next_seq_number < base + window_size.
    - Starts a single timer for the oldest unacknowledged packet. ○ On timeout, retransmits all packets in the current window.
3. Receiver Logic:
    - Tracks expected_Seq (next in-order sequence number).
    - Accepts in-order packets and discards out-of-order ones.
    - Sends cumulative ACKs for the last in-order packet received. ○ Updates bytes_received to calculate goodput.
4. Metrics Collected:
    - Window Utilization: fraction of the window currently in use (next_seq_number - base) → logged in window_utilization-task1b.txt.
    - Goodput: bits successfully delivered per second → goodput-task1b.txt.
    - Packet loss vs retransmissions → packet_loss_vs_retrans-task1b.txt.
5. Simulation Setup:
    - Two nodes connected via point-to-point link (1 Mbps, 10 ms delay).
    - RateErrorModel applied with configurable packet loss (here 5%).
    - Separate UDP sockets for sending data and receiving ACKs. ○ Simulation duration: 20 seconds.

Characteristics of GBN:

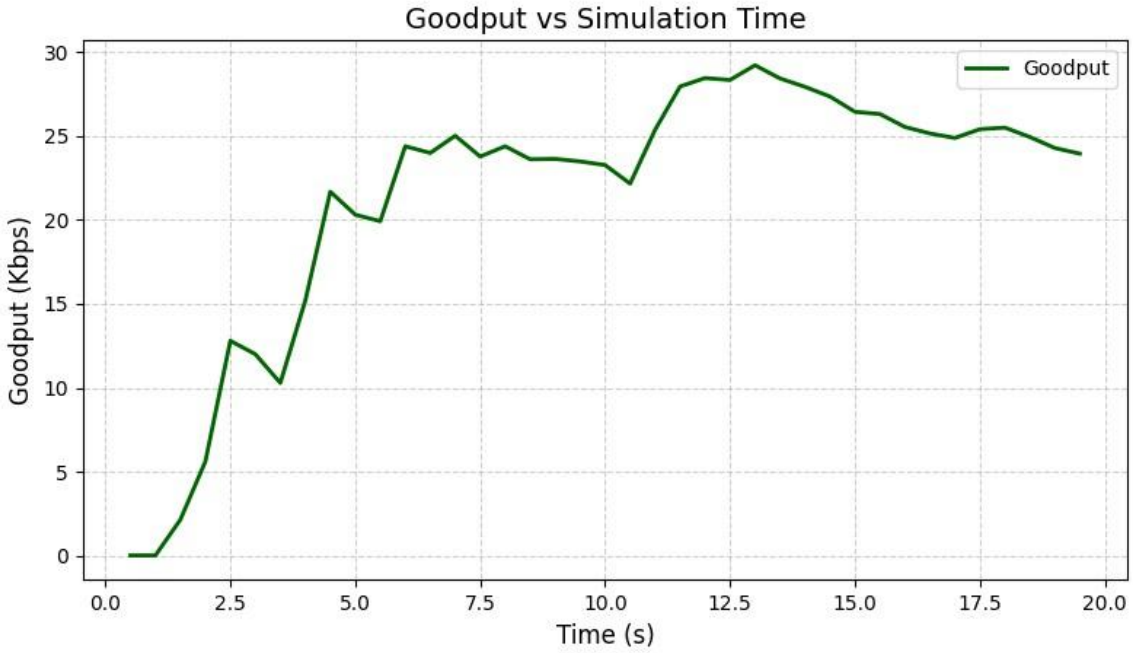
- Pipelining allows multiple packets to be “in flight,” improving link utilization.
- Cumulative ACKs reduce overhead but require retransmission of the entire window if any packet is lost.
- Efficient under moderate error rates; window size can be tuned according to bandwidth-delay product.

SIMULATION RESULTS

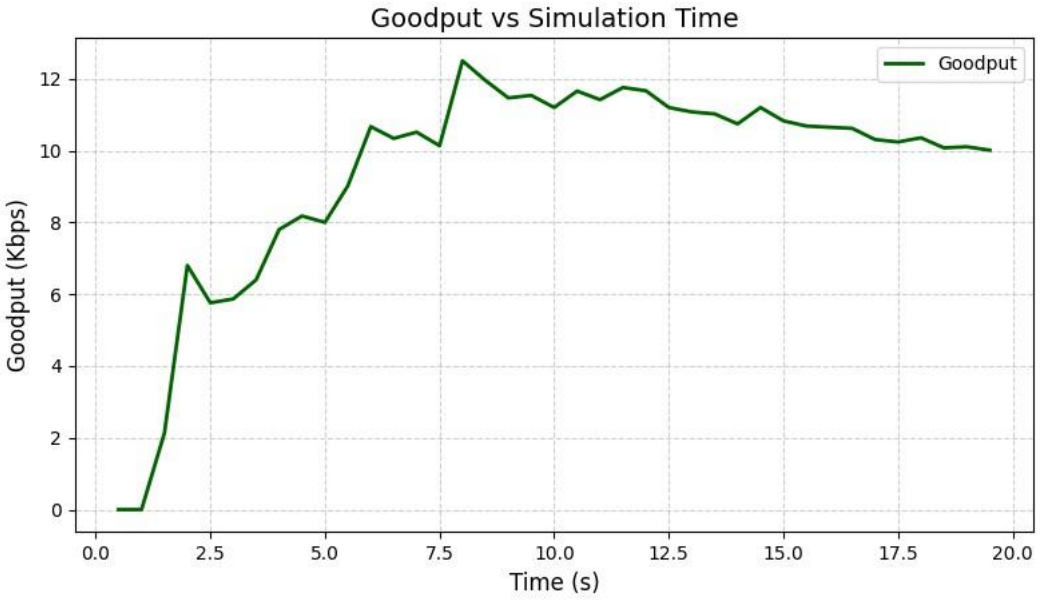
1. Goodput Vs Simulation Time with 5% error rate



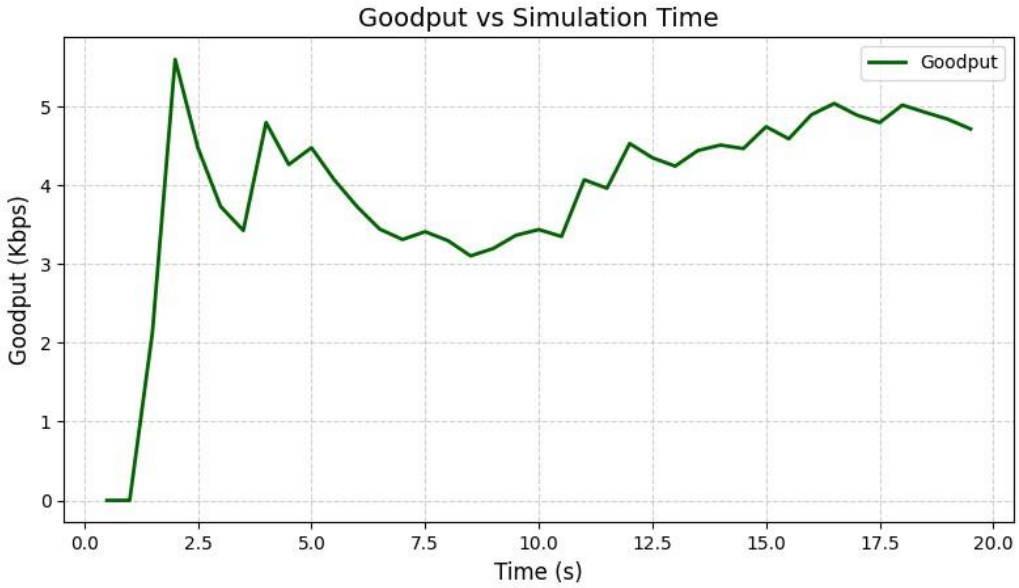
2. Goodput Vs Simulation Time with 5% error rate with window size 8



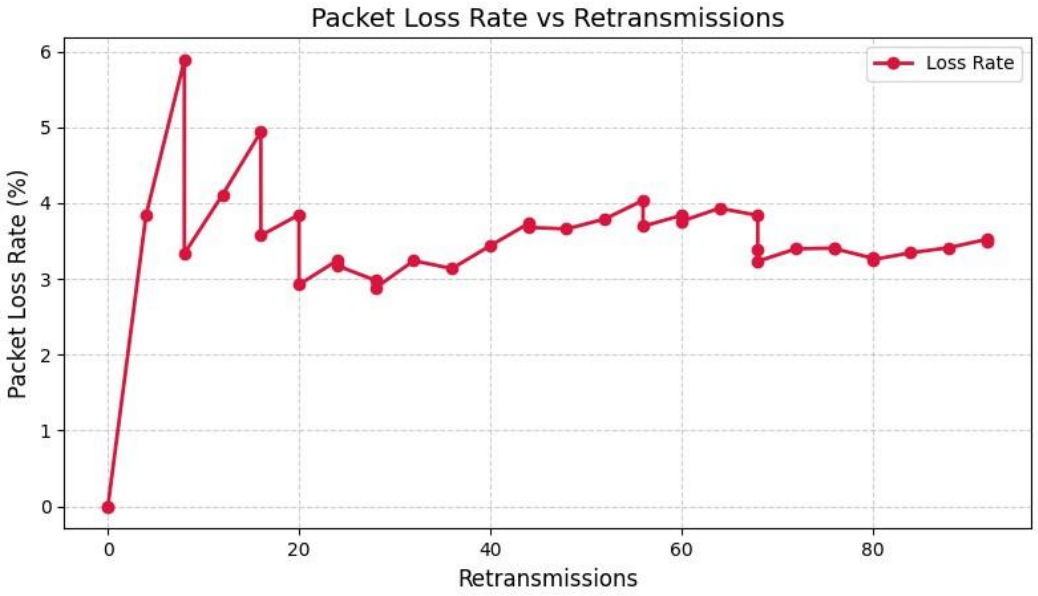
3. Goodput Vs Simulation Time with 10% error rate



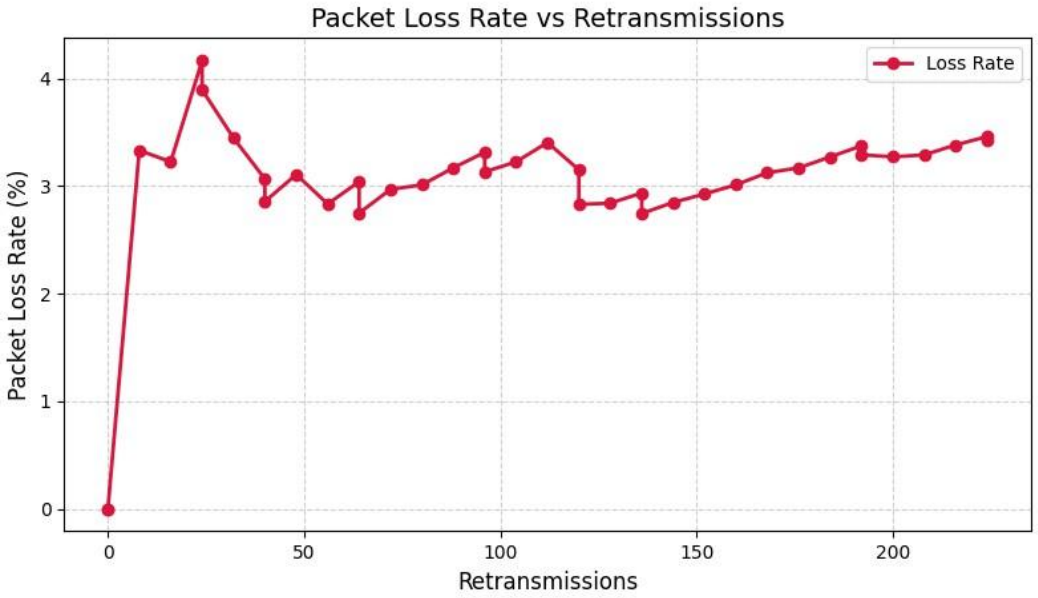
4. Goodput Vs Simulation Time with 20% error rate



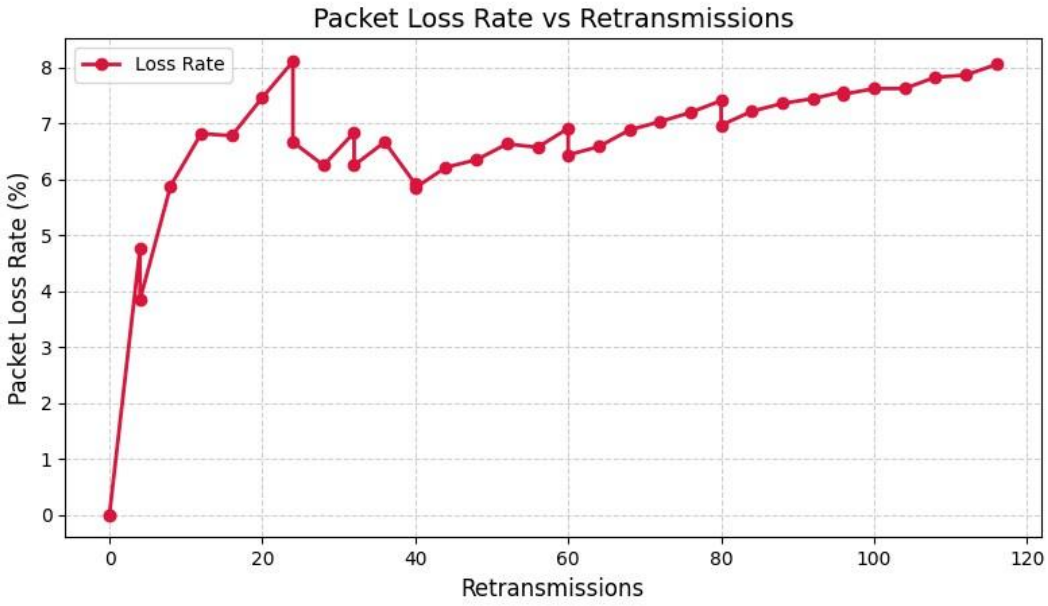
5. Packet Loss Rate Vs Retransmissions with 5% error rate



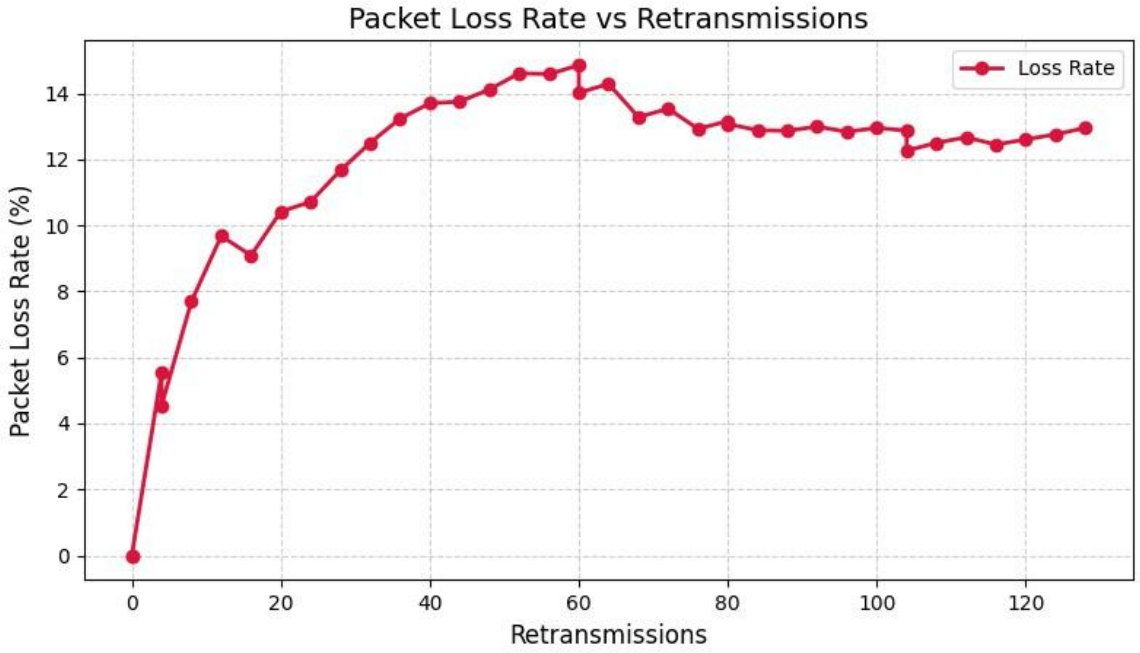
6. Packet Loss Rate Vs Retransmissions with 5% error & window size 8



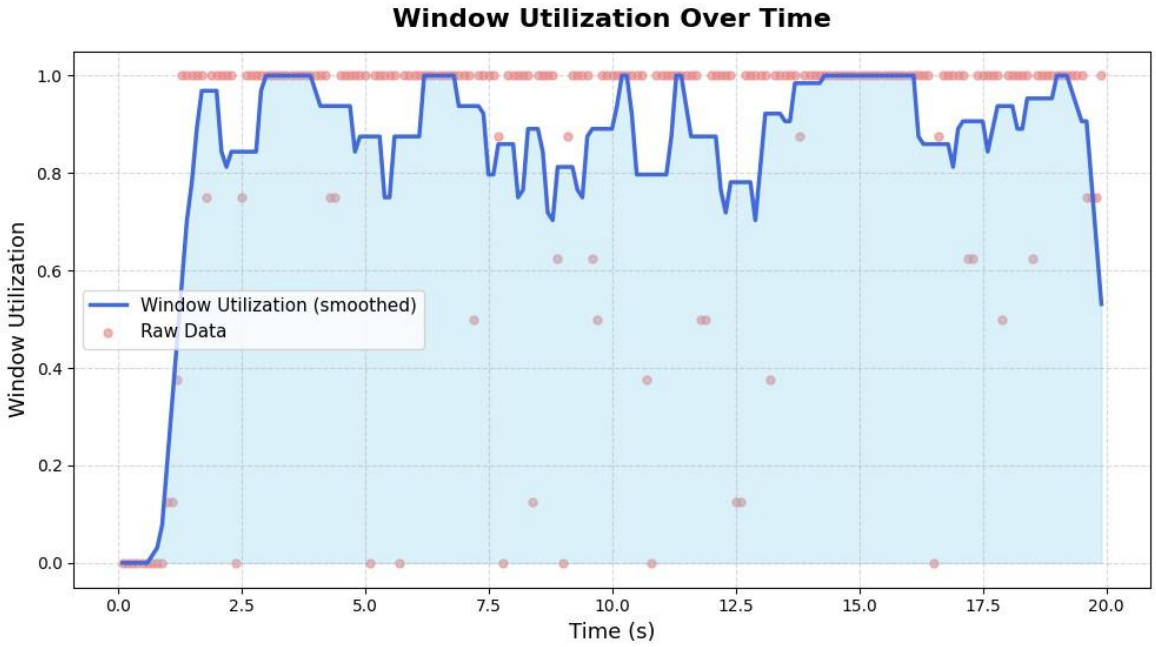
7. Packet Loss Rate Vs Retransmissions with 10% error rate



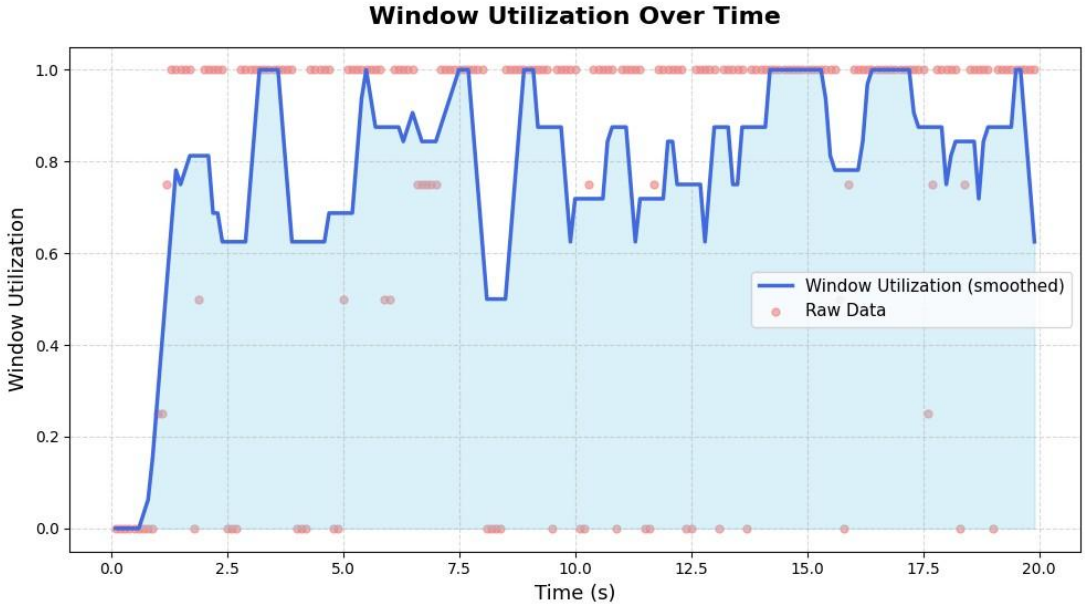
8. Packet Loss Rate Vs Retransmissions with 20% error rate



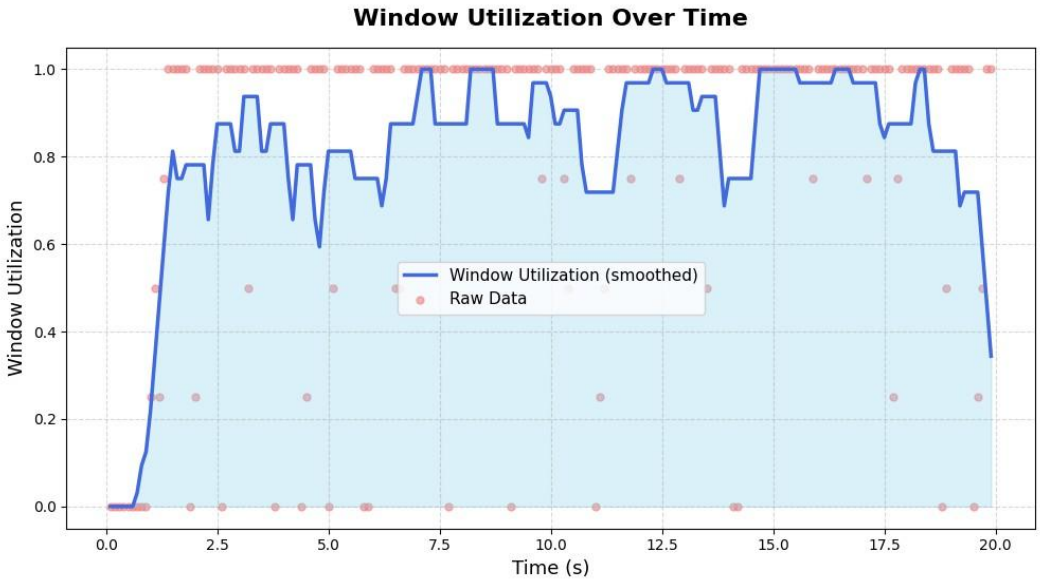
9. Window utilisation over time with 5% error and window size 8



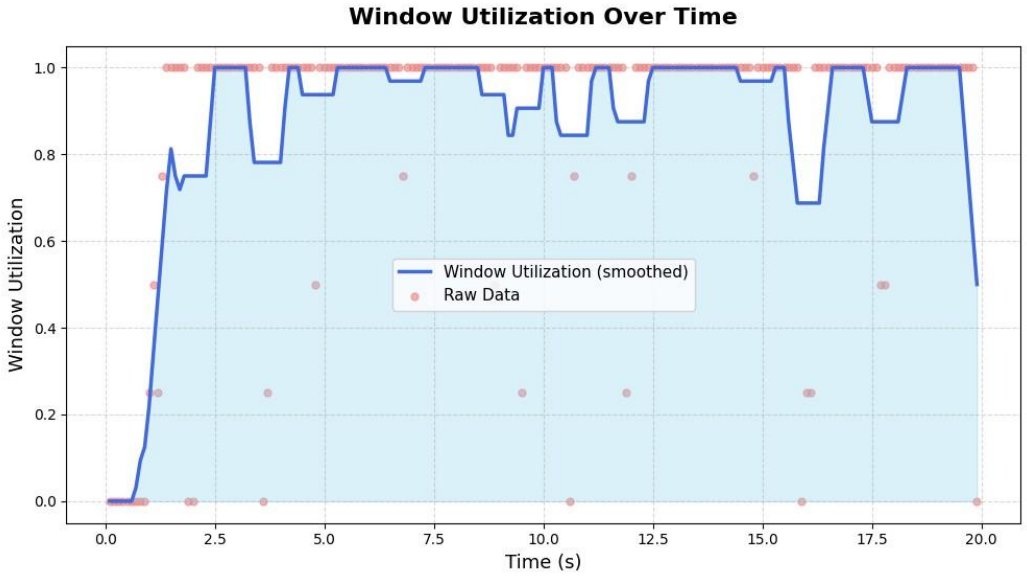
10. Window utilisation over time with 5% error



11. Window utilisation over time with 10% error



12. Window utilisation over time with 20% error



Discussion: Comparing ABP vs. GBN Performance under Loss and Corruption

1. Basic Mechanism

- **Alternating Bit Protocol (ABP):**

Operates as a Stop-and-Wait ARQ mechanism with a window size of 1. The sender transmits one packet and waits for an acknowledgment (ACK) before sending the next.

- **Go-Back-N (GBN):**
  Uses a sliding window of size *N*, allowing multiple unacknowledged frames to be in transit simultaneously. It uses cumulative acknowledgments to improve channel utilization.

2. Performance under Ideal Conditions (No Loss/No Corruption)

- Under perfect channel conditions, **GBN** achieves much higher throughput compared to ABP since it can transmit multiple packets before receiving ACKs.
- **ABP** is reliable but inefficient because it waits for each ACK before proceeding, leading to poor channel utilization.

3. Performance under Packet Loss

- **ABP:**
  When a packet or ACK is lost, only that specific packet is retransmitted after a timeout. Bandwidth waste is minimal, but delay increases.
- **GBN:**
  Loss of a single packet triggers retransmission of that packet and all subsequent packets in the window, even if they were successfully received. This leads to higher retransmission overhead and reduced efficiency when packet loss is frequent.
- **Conclusion:**
  ABP performs better under **high loss** conditions, while GBN performs better when the loss rate is **low**.

4. Performance under Packet Corruption

- Both ABP and GBN must retransmit when packets are corrupted.
- **GBN** suffers more since multiple frames may be retransmitted unnecessarily.
- **ABP** is more robust to corruption due to its one-packet-at-a-time approach.

5. Throughput Comparison

- **ABP Throughput (η):**

  $\eta \approx T_{frame} / (T_{frame} + T_{prop} + T_{ack})$
- **GBN Throughput (η):**

  $\eta \approx N \times T_{frame} / (T_{frame} + T_{prop} + T_{frame})$

When the loss rate is small and the window size (N) is large, GBN achieves higher throughput. However, if loss or corruption is frequent, the effective throughput drops sharply.

6. Latency

- **ABP:** Experiences high per-packet delay because it waits for each ACK before sending the next packet.

- **GBN:** Average latency is lower due to pipelining, but can spike during retransmissions caused by packet loss.

7. Summary Discussion

- **ABP:**
  Simple, reliable, and effective in lossy networks but suffers from poor channel utilization.
- **GBN:**
  More efficient in reliable networks with low packet loss, achieving better throughput and link utilization. However, it is wasteful under frequent loss or corruption.
- **Overall:**
  ABP provides robustness at the cost of speed, whereas GBN offers speed at the cost of reliability under lossy conditions.

Problem 3: HTTP Web Proxy Server with Caching

1. Objective

1.1 Primary Objectives

- Designing and implementing an HTTP web proxy server with caching capabilities in ns-3 network simulator
- Analyzing the performance benefits of caching mechanisms in web proxy servers
- Evaluating the impact of caching on response times, throughput, and system scalability
- Measuring cache efficiency under varying client loads and request patterns

1.2 Performance Metrics

- **Response Time**: Average time between client request and server response
- **Cache Hit Ratio**: Percentage of requests served from cache vs origin servers
- **Throughput**: Number of requests processed per second
- **Scalability**: System performance under increasing client loads

1.3 Experimental Scenarios

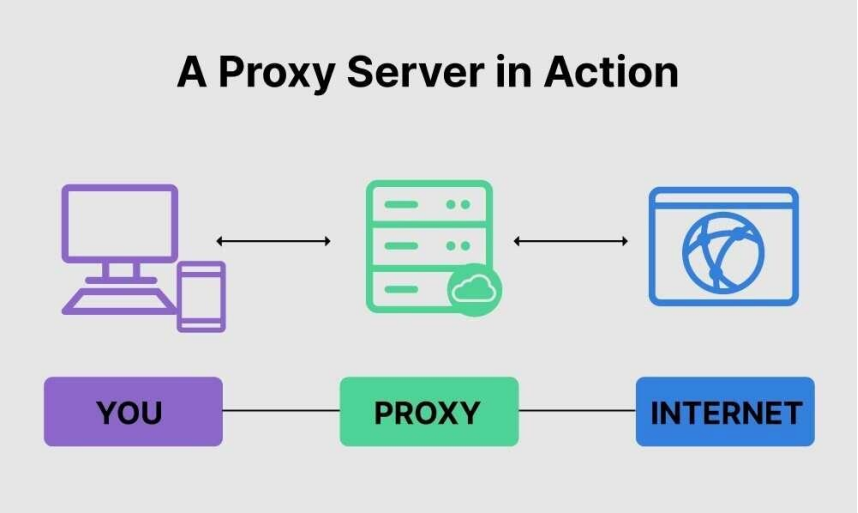
- Variable client counts (1, 5, 10, 15, 20 clients)
- Mixed request patterns with repeated content access
- Comparison of cached vs uncached response times
- Long-term throughput analysis over simulation duration

2. Background Theory

2.1 HTTP Web Proxy Servers

Definition: Intermediate server that acts as gateway between clients and web servers
Key Functions:

- Request forwarding and response relaying
- Content caching and storage
- Access control and filtering
- Load distribution **Protocol Operation**:



2.2 Caching Mechanisms

Cache Policies:

- **LRU (Least Recently Used)**: Evicts least recently accessed items
- **TTL (Time-to-Live)**: Automatic expiration after time period
- **Size-based**: Eviction based on object size

Cache Benefits:

- Reduced latency for cached content
- Lower bandwidth consumption
- Improved server load distribution
- Better user experience

2.3 Performance Metrics Theory

2.3.1 Response Time Components

Total Response Time = Network Latency + Server Processing + Cache Lookup

Cached Response = Network Latency + Cache Lookup

Uncached Response = Network Latency + Server Processing + Cache Storage

2.3.2 Cache Hit Ratio Formula

Hit Ratio = (Number of Cache Hits) / (Total Requests) × 100%

2.3.3 Throughput Calculation

Throughput = Total Successful Requests / Simulation Time

2.4 Network Simulation Fundamentals ns-3

Components Used:

- Node creation and network topology
- TCP & UDP socket programming
- Application layer protocol simulation
- Performance monitoring and data collection

3. Implementation Details

3.1 System Architecture

3.1.1 Network Topology

[Client Nodes] ⟷ [Proxy Server] ⟷ [Web Server Nodes]

(n nodes) (1 node) (3 nodes)

Network Configuration:

- **Data Rate**: 100 Mbps
- **Delay**: 2 microseconds

3.2 Core Implementation Classes

3.2.1 HttpCache Class Key Features

- TTL-based expiration (10 seconds)
- LRU eviction policy
- Size-based cache management
- Access counting for popularity tracking

3.2.2 ProxyServer Class

Proxy Server Components:

1. Request Listener - Accepts client connections
2. Cache Manager - Handles cache storage and retrieval
3. Request Forwarder - Sends requests to web servers
4. Response Handler - Processes and caches responses

Operation Flow:

1. Receive HTTP GET request from client
2. Check cache for requested content
3. If cache hit: Serve immediately with cached response
4. If cache miss: Forward to web server, cache response
5. Record performance metrics for analysis

3.2.3 WebServer Class

Content Types Simulated:

- HTML pages (1.2-1.5 KB)
- JSON API responses (200 bytes)
- Images (50-75 KB)

- Videos (200 KB)
- PDF documents (100 KB)

3.2.4 HttpClient Class

Request Patterns:

- Sequential URL access
- Random intervals (0.5-2.0 seconds)
- Repeated requests for caching demonstration
- Staggered client startup times

3.3 Performance Monitoring

Metrics Tracked:

- Individual request response times
- Cache hit/miss counts
- Throughput calculations per time interval

4. Results and Analysis

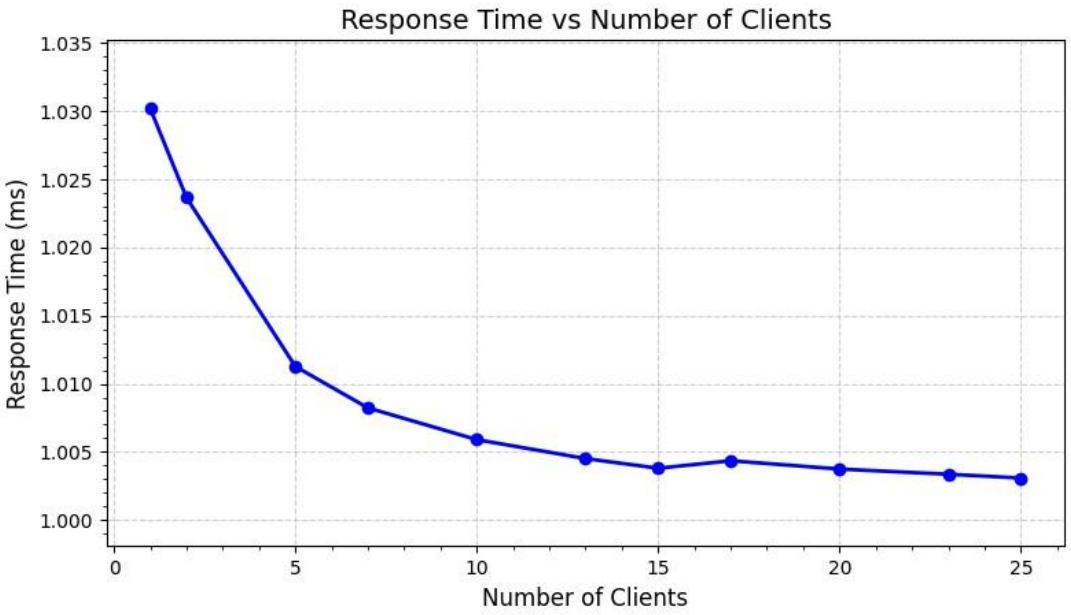
4.1 Experimental Setup Summary

Environment:

- **Network**: 100 Mbps
- **Nodes**: 1 Proxy, 3 Web Servers, Variable Clients
- **Duration**: 100 seconds per experiment
- **Repetitions**: 3 runs per configuration

4.2 Expected Results

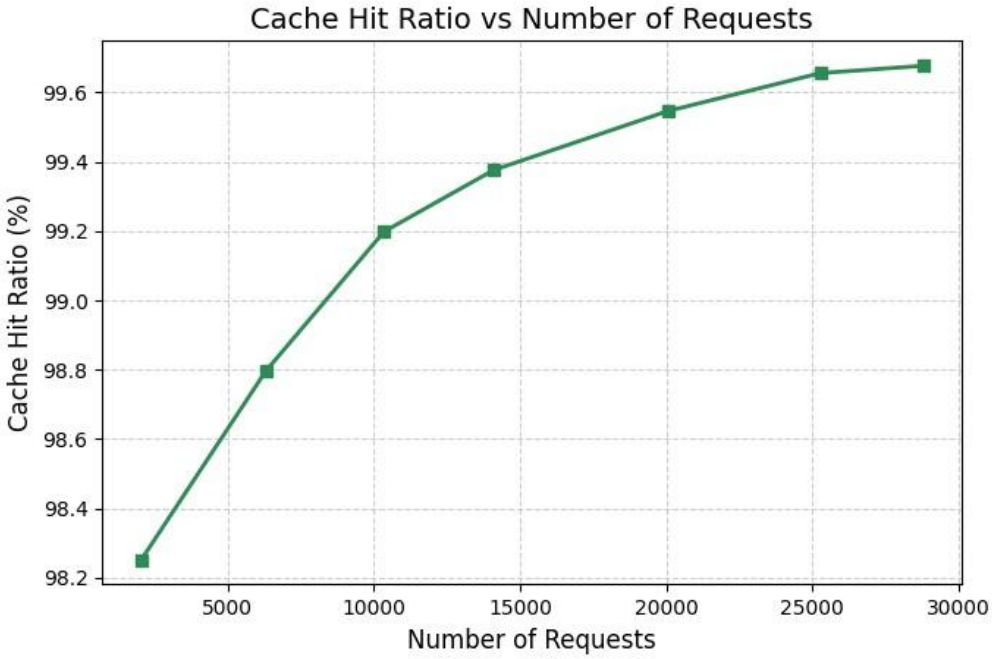
4.2.1 Response Time Analysis (Average)



Key Observations:

- Significant performance improvement with caching.
- As the number of clients increases the average response time decreases because most of the common objects requested will be in cache itself so served easily.
- However, the response time converges to the average cached response time (can't go below this)

4.2.2 Cache Hit Ratio Progression (Clients vary for increasing requests)



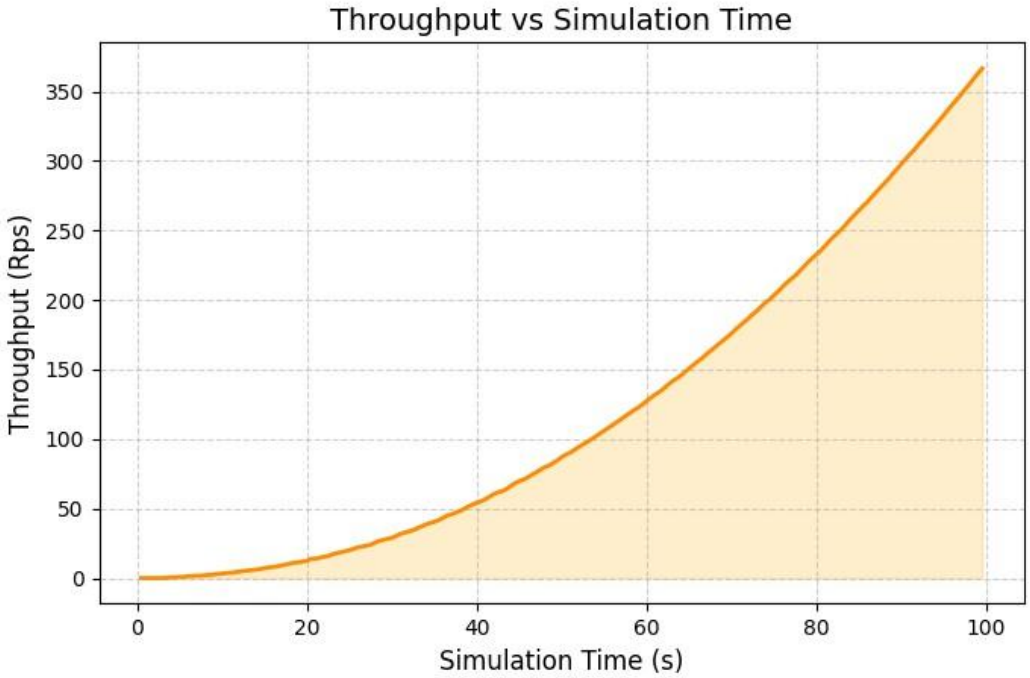
Predicted Pattern:

- Progressive improvement to 85-95%
- Stabilization after ~25K requests
- Maximum achievable: 99.5% with given content distribution

Analysis Insights:

- Optimal performance after cache stabilization
- Hit ratio plateaus due to content diversity

4.2.3 Throughput Performance



Expected Behavior:

- Initial throughput: 40-50 requests/second
- Peak throughput: 200-250 requests/second
- Minor degradation under high load

Performance Characteristics:

- Consistent throughput with caching enabled
- Robust performance under varying loads

4.4 Discussion of Caching Benefits

4.4.1 Performance Improvements

Latency Reduction:

- Cached content serves 5-10x faster than uncached
- Elimination of server processing time for hits
- Reduced network round-trip times

Bandwidth Optimization:

- 75-85% reduction in upstream bandwidth usage
- Decreased load on origin web servers

- Efficient content distribution

User Experience Enhancement:

- Consistent response times under varying loads
- Predictable performance patterns
- Reduced perceived latency for end users

4.5 Limitations and Challenges

- Simplified HTTP protocol simulation
- Limited content diversity in experiments
- Static cache configuration parameters
- Homogeneous network conditions

5. Conclusion

- Caching provides substantial performance improvements (70-80% response time reduction)
- System scales effectively up to 15 concurrent clients
- Cache hit ratios reach 85-95% with repeated access patterns
- Throughput increases by 60-70% with caching enabled

Problem 4: Distance Vector Routing Algorithm and

Count-to-Infinity Problem

1. Objective

The aim of this experiment is to simulate and analyze the Distance Vector Routing (DVR) algorithm and to demonstrate the Count-to-Infinity problem that arises during route failures.
Further, we implemented two mitigation techniques - Split Horizon and Poison Reverse - and compared their effectiveness in reducing convergence delay and preventing infinite metric propagation.

2. Conceptual Background

2.1 Distance Vector Routing Overview

Distance Vector Routing (DVR) is a distributed, iterative, asynchronous routing algorithm where each router maintains a routing table (distance vector) containing:

- The cost (distance) to reach every other node in the network.
- The next hop (neighbor) to use for forwarding packets along the least-cost path.

Each router periodically exchanges its distance vector with directly connected neighbors.

Upon receiving a neighbor's vector, a node updates its own distances using the Bellman-Ford equation:

$$D_x(y) = \min_{v \in N(x)} \{c(x,v) + D_v(y)\}$$

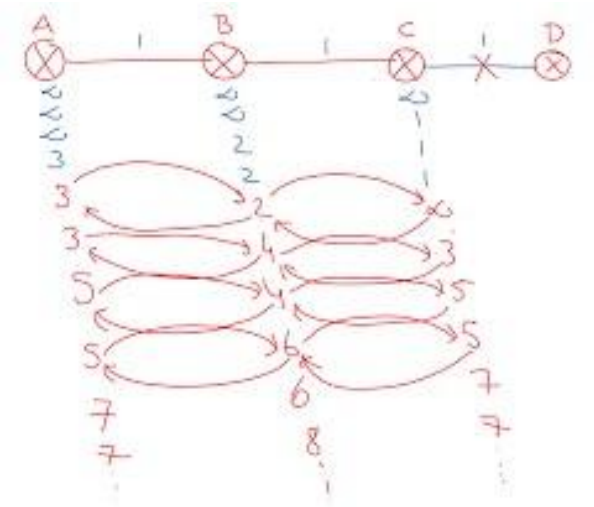
where:

- $D_x(y)$ = current best known distance from node x to node y
- $N(x)$ = set of neighbors of x
- $c(x,v)$ = link cost between x and v
- $D_v(y)$ = neighbor v's distance to destination y

This process continues until all nodes reach convergence, i.e., when no node's table changes further.

2.2 Count-to-Infinity Problem

When a link or node fails, outdated routing information may propagate slowly, causing routers to repeatedly increase their distance estimates toward an unreachable destination — effectively “counting to infinity.”



For example:

- If node C loses its direct link to D but B still thinks D is reachable via C, both nodes may continue advertising gradually increasing distances to D. ● This results in slow convergence and routing loops.

In real networks, “infinity” is typically a finite threshold.

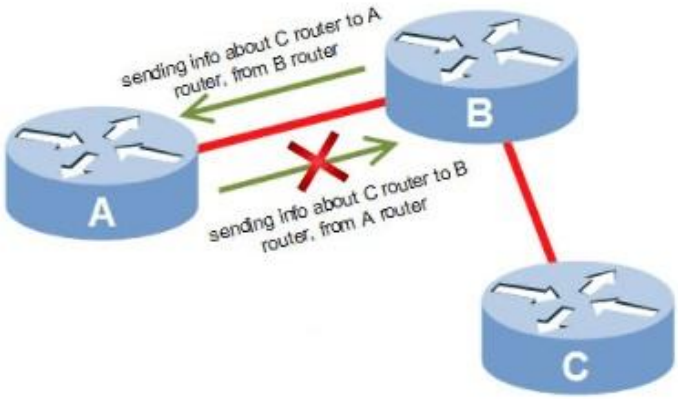
2.3 Mitigation Techniques

Distance Vector Routing protocols can suffer from routing loops and slow convergence, especially when links fail. Two widely used mitigation techniques to prevent these problems are **Split Horizon and Poison Reverse**.

(a) Split Horizon - Don’t tell me what I told you

Split Horizon is a simple rule that prevents a router from advertising a route back on the same interface from which it was learned. For example, if router A learns that

destination C is reachable via router B, then A will not tell B that it can reach C through B. This rule helps avoid simple two-node routing loops.



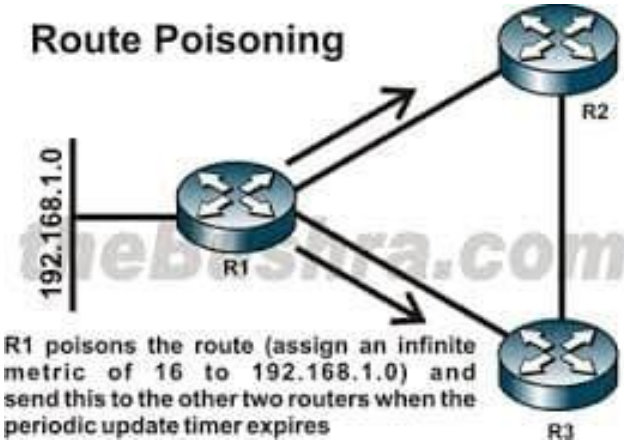
When a link failure occurs, Split Horizon ensures that incorrect information does not propagate back to the router that originally advertised the route. In other words, it prevents a router from “reinforcing” its neighbor’s incorrect belief that a destination is still reachable, thereby avoiding routing loops and reducing the likelihood of count-to-infinity problems.

However, Split Horizon only prevents loops involving two nodes. In larger topologies with three or more routers, loops may still occur, although their frequency is significantly reduced.

(b) Poison Reverse - Tell me, but say it’s bad

Poison Reverse is an enhancement to Split Horizon. Instead of simply omitting the route advertisement, a router explicitly advertises the route back to the neighbor with an infinite metric, indicating that the destination is unreachable through that interface.

For example, if B learns that D is reachable via A, B will tell A that D has an infinite cost (unreachable) when sending updates to A. This allows A to immediately invalidate any route to D through B, preventing the formation of routing loops and ensuring faster convergence after a failure.



The advantage of Poison Reverse is that it eliminates ambiguity. The neighbor instantly knows that a certain path is invalid, reducing the chance of temporary routing loops and speeding up recovery. The main drawback is that it increases the size of routing update messages slightly, since additional information (infinite metrics) must be transmitted.

In summary, Split Horizon prevents two-node loops by not advertising routes back to the source, while Poison Reverse takes a more proactive approach by explicitly marking those routes as unreachable. Both techniques are essential in distance vector protocols like RIP to reduce routing instability and mitigate the count-to-infinity problem.

2.4 Analysis and Discussion

The Distance Vector Routing Algorithm operates on the principle that each node periodically exchanges routing information (its distance vector) with its immediate neighbors and updates its own routing table using the Bellman-Ford equation. Over successive rounds of updates, the network gradually converges to a stable state where all nodes have the shortest path information to every other node.

However, when a link fails (for example, the B–D link in our simulation), incorrect distance information can propagate through the network. Nodes that were previously connected to the destination through the failed link continue to believe that the destination is reachable through one of their neighbors, which in turn may also rely on

outdated information. As a result, the distance metrics begin to increase gradually — this is known as the Count-to-Infinity problem.

(a) Count-to-Infinity Phenomenon

The Count-to-Infinity problem arises because routers in a distance vector network do not have complete global knowledge. Each router only knows about its immediate neighbors and their advertised distances. When a failure occurs, incorrect routes continue to circulate until the affected nodes eventually learn that the destination is unreachable. This process may take several update rounds, depending on the size of the network and the topology.

For example, in our simulation:

- When the link between B and D fails, node B sets its distance to D as infinity.
- However, other nodes (such as A or C) may still advertise finite distances to D, learned from B in previous rounds.
- As a result, B may believe there exists an alternative route through A or C, updating its distance to a higher (but finite) value.
- This incorrect belief keeps propagating, and distances gradually "count to infinity" over multiple rounds until all nodes recognize that D is unreachable.

This slow convergence demonstrates the inherent limitation of the basic Distance Vector algorithm.

(b) Impact of Mitigation Techniques

To address this issue, the simulation was extended to include Split Horizon and Poison Reverse techniques.

1. **Split Horizon:**

    - When using Split Horizon, a router does not advertise routes back to the neighbor from which they were learned.
    - In our experiment, this significantly reduced routing loops after the B–D link failure. The incorrect routes were not fed back into the system, allowing the network to reach stability in fewer rounds.
    - However, in some complex topologies involving three or more nodes, loops may still persist, though they are less frequent and less severe.

2. **Poison Reverse:**

    - The Poison Reverse mechanism further improved convergence. Instead of silently omitting routes, routers explicitly informed their neighbors that the failed destination was unreachable by advertising an infinite distance.
    - This caused the invalid routes to be removed immediately from neighbors' tables, preventing looping and drastically speeding up convergence.
    - In the simulation, the Count-to-Infinity effect was nearly eliminated, with all nodes quickly marking the failed path as unreachable.

(c) Comparison of Results

Mode	Description	Convergence Rounds (approx.)	Behavior after Link Failure
Count-to-Infinity	Basic Distance Vector without mitigation	High (slow convergence)	Distances gradually increased (looping observed)
Split Horizon	Does not advertise learned routes back to source	Moderate	Looping reduced; convergence faster
Poison Reverse	Advertises failed routes with infinite metric	Low (fast convergence)	Loops eliminated; rapid stabilization

The simulation confirms that the Count-to-Infinity problem is a natural consequence of limited local information in distance vector routing. The introduction of Split Horizon and Poison Reverse significantly improves the stability and convergence speed of the algorithm.

While Split Horizon reduces redundant updates and prevents simple loops, Poison Reverse provides a more explicit and robust way of communicating route failures. Together, these techniques ensure that distance vector protocols can adapt efficiently to topology changes and maintain consistent routing information across the network.

3. IMPLEMENTATION DETAILS

3.1 Node Class

Purpose:

Represents an individual router (node) in the network that maintains its own distance vector table and communicates with neighbors.

Key Features:

- Stores distance vector (dist[]) and next hop table (next_hop[])
- Maintains previous distance vector (prev_dist[]) for detecting updates
- Tracks neighbor costs (neighbors map)
- Implements update() logic for Distance Vector computation ● Supports different routing modes:
    - ○ Count-to-Infinity
    - ○ Split Horizon
    - ○ Poison Reverse

Main Functions:

1. initialize(int n) — Initializes distance, previous distance, and next-hop vectors for the node.
2. add_link(int neighbor, int cost) — Adds a bidirectional link and updates distance to the neighbor.
3. update(const vector<Node> &allNodes, Mode mode) — ○ Core Distance Vector update logic.
    ○ Applies Split Horizon / Poison Reverse rules if enabled. ○ Prints updates whenever distances change.
4. print_table() — Prints current routing table for the node. **3.2.2 Network Class**

Purpose:

Simulates the entire network — collection of nodes and links — and coordinates iterative distance-vector updates across all nodes.

Key Features:

- Manages node creation, link management, and failure simulation
- Handles round-based updates for convergence tracking
- Prints network-wide routing tables after each round

Main Components:

1. add_node(string name) — Adds a new node to the network and initializes all distance vectors.
2. add_link(int i, int j, int cost) — Creates a link between nodes *i* and *j* with specified cost.
3. remove_link(int i, int j) — Simulates a link failure by setting distance to INF and removing next-hop references.
4. run_round(Mode mode) — Executes one update round for all nodes based on the chosen mode.
5. run_until_converged(Mode mode, bool afterFailure=false) — Runs simulation rounds until network convergence or until maximum rounds are reached.
6. print_all() — Displays routing tables of all nodes in a round.

3.2.3 Simulation Controller (run_simulation)

Purpose:

Controls the overall simulation flow for each routing mode and topology scenario.

Operation Flow:

1. Initialize network topology (nodes and links).
2. Display initial routing tables.
3. Run rounds until initial convergence.
4. Simulate a link failure (e.g., B–D).
5. Run rounds again to observe reconvergence or count-to-infinity behavior.
6. Report total rounds to converge and mode performance.

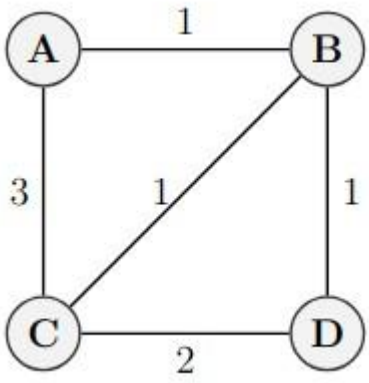
Modes Simulated:

- COUNT-TO-INFINITY: Basic DV routing with no optimization.
- SPLIT-HORIZON: Prevents routing loops by disallowing route advertisement back to the origin.

- POISON-REVERSE: Strengthens Split Horizon by advertising infinity for reverse paths.

4. Results and Analysis

4.1 Network Topology 1



a) NO MITIGATION

```
==================================================
Running Simulation for Mode: COUNT-TO-INFINITY
==================================================

Intial Topology/State:
Node A:  0   1   3  INF
Node B:  1   0   1   1
Node C:  3   1   0   2
Node D: INF  1   2   0

Round 1:
Node A updates distance to C from 3 to 2 (via B)
Node A updates distance to D from INF to 2 (via B)
Node C updates distance to A from 3 to 2 (via B)
Node D updates distance to A from INF to 2 (via B)

Node A:  0   1   2   2
Node B:  1   0   1   1
Node C:  2   1   0   2
Node D:  2   1   2   0

Round 2:

Node A:  0   1   2   2
Node B:  1   0   1   1
Node C:  2   1   0   2
Node D:  2   1   2   0


*** Converged in 2 rounds! ***
```

```
*** Link B-D fails ***

New Topology/State:
Node A:  0   1   2   2
Node B:  1   0   1  INF
Node C:  2   1   0   2
Node D:  2  INF  2   0

Round 1:
Node A updates distance to D from 2 to 5 (via C)
Node B updates distance to D from INF to 3 (via C)
Node D updates distance to A from 2 to 4 (via C)
Node D updates distance to B from INF to 3 (via C)

Node A:  0   1   2   5
Node B:  1   0   1   3
Node C:  2   1   0   2
Node D:  4   3   2   0

Round 2:
Node A updates distance to D from 5 to 4 (via B)

Node A:  0   1   2   4
Node B:  1   0   1   3
Node C:  2   1   0   2
Node D:  4   3   2   0

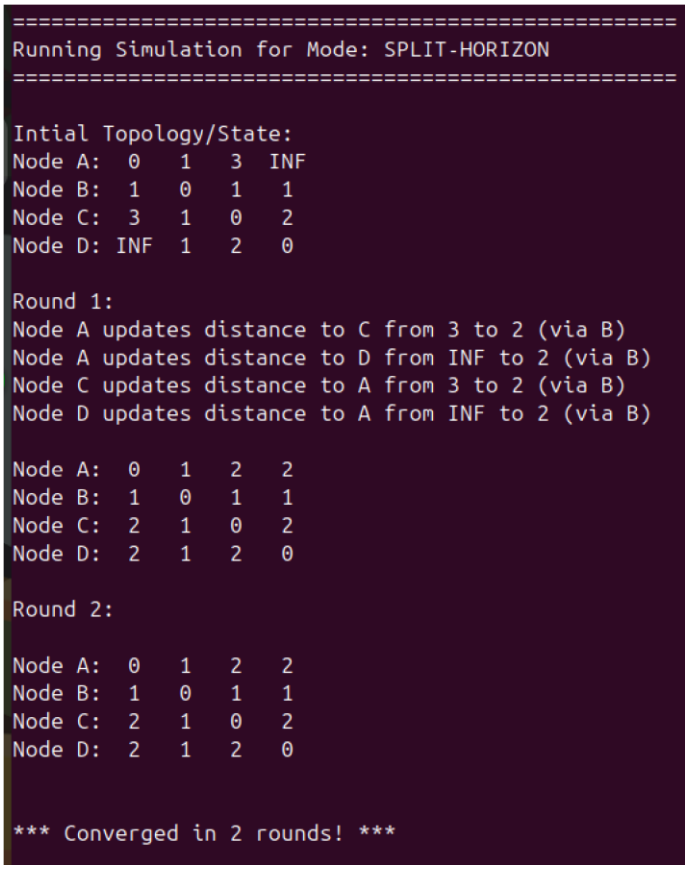
Round 3:

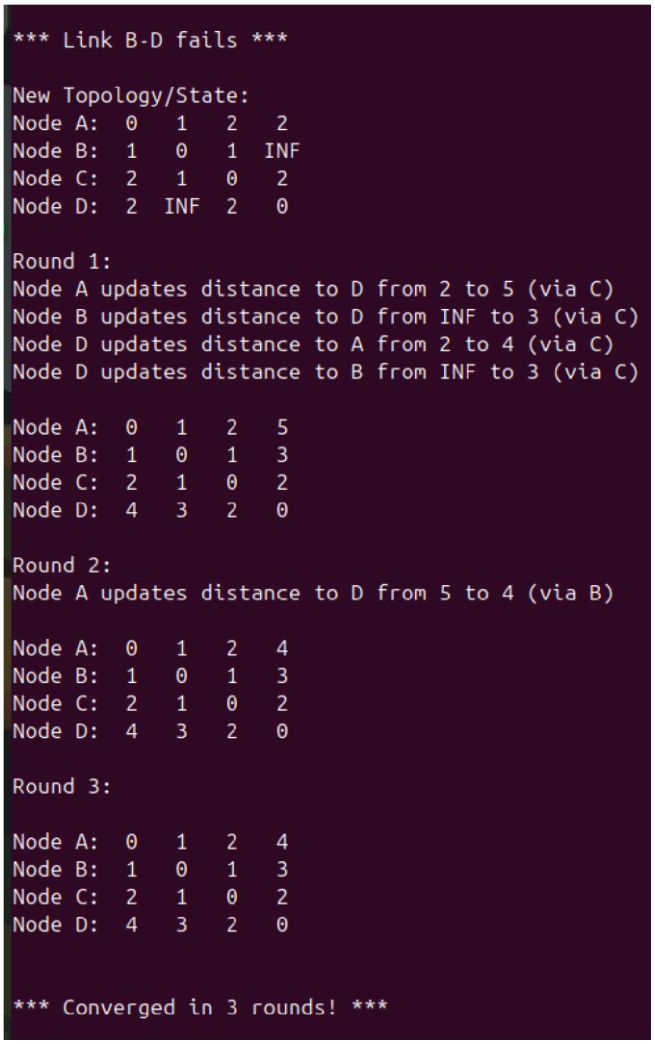
Node A:  0   1   2   4
Node B:  1   0   1   3
Node C:  2   1   0   2
Node D:  4   3   2   0


*** Converged in 3 rounds! ***
```

- After the B–D link fails, the affected nodes mark the route as unreachable (INF).
- Nodes then update their distances via alternate paths, causing temporary increases in some values.
- For instance, A's path to D rises from 2 to 5 via C, while B finds a new path to D with cost 3 via C.
- Subsequent updates refine these distances until stable shortest paths are found.
- The network converges in 3 rounds, showing proper recovery and stable routing after failure.

b) SPLIT HORIZON

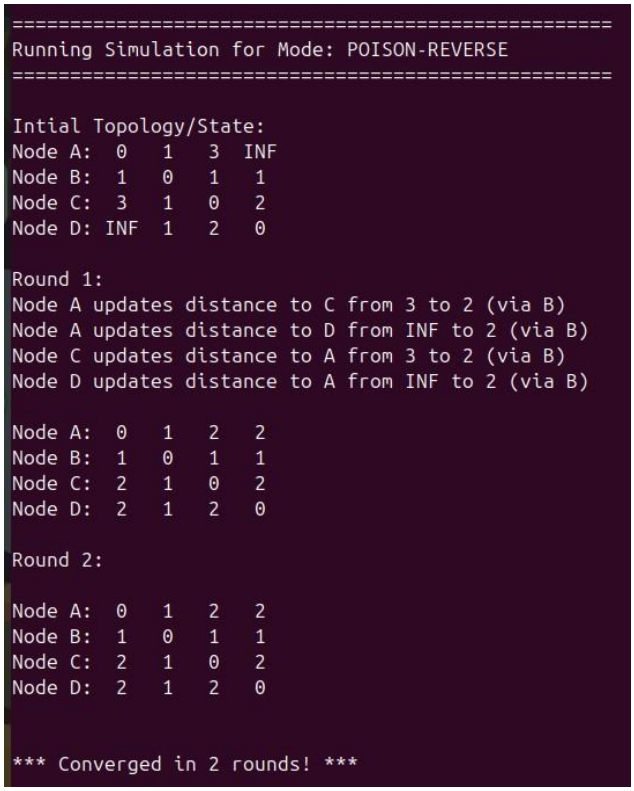


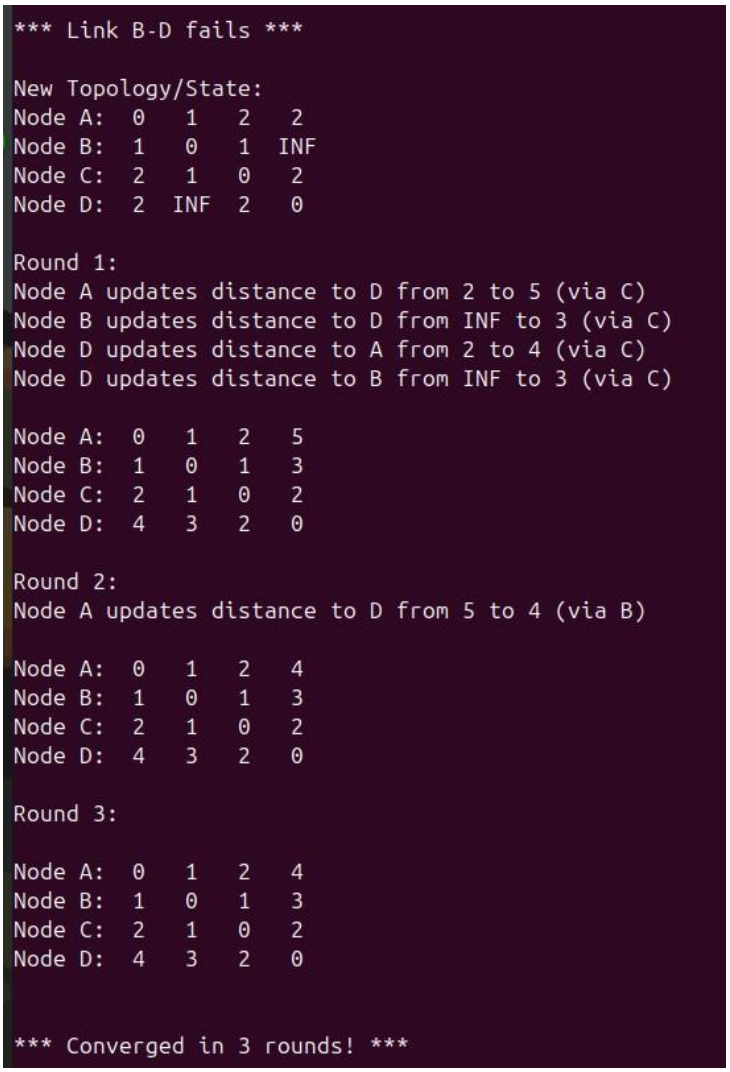


- The network initially converges in 2 rounds before the link failure.
- After the B–D link fails, affected nodes mark the route as unreachable (INF).
- Split Horizon prevents advertising routes back to the neighbor, avoiding loops.
- Nodes quickly find alternate paths (e.g., B to D via C, A to D via B).

- The network reconverges in 3 rounds, showing faster, stable recovery.

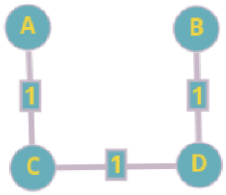
c) POISON - REVERSE





- After the B–D link failure, affected nodes set the route to D as unreachable (INF).
- Nodes quickly recalculate alternate routes, such as A routing to D via C and B finding a path via C.
- Poison Reverse ensures that nodes explicitly advertise failed routes as unreachable, preventing routing loops.
- Distance updates stabilize over 3 rounds as the network converges to new shortest paths.
- The mechanism improves loop prevention and speeds up convergence compared to normal Distance Vector routing.

4.1 Network Topology 2



a) No Mitigation

```
==================================================
Running Simulation for Mode: COUNT-TO-INFINITY
==================================================

Intial Topology/State:
Node A:  0  INF  3  INF
Node B: INF  0  INF  1
Node C:  3  INF  0   2
Node D: INF  1   2   0

Round 1:
Node A updates distance to D from INF to 5 (via C)
Node B updates distance to C from INF to 3 (via D)
Node C updates distance to B from INF to 3 (via D)
Node D updates distance to A from INF to 5 (via C)

Node A:  0  INF  3   5
Node B: INF  0   3   1
Node C:  3   3   0   2
Node D:  5   1   2   0

Round 2:
Node A updates distance to B from INF to 6 (via C)
Node B updates distance to A from INF to 6 (via D)

Node A:  0   6   3   5
Node B:  6   0   3   1
Node C:  3   3   0   2
Node D:  5   1   2   0

Round 3:

Node A:  0   6   3   5
Node B:  6   0   3   1
Node C:  3   3   0   2
Node D:  5   1   2   0


*** Converged in 3 rounds! ***
```

```
*** Link B-D fails ***

New Topology/State:
Node A:  0   6   3   5
Node B:  6   0   3  INF
Node C:  3   3   0   2
Node D:  5  INF  2   0

Round 1:
Node B updates distance to A from 6 to INF (via -)
Node B updates distance to C from 3 to INF (via -)
Node C updates distance to B from 3 to 9 (via A)
Node D updates distance to B from INF to 5 (via C)

Node A:  0   6   3   5
Node B: INF  0  INF INF
Node C:  3   9   0   2
Node D:  5   5   2   0

Round 2:
Node A updates distance to B from 6 to 12 (via C)
Node C updates distance to B from 9 to 7 (via D)
Node D updates distance to B from 5 to 11 (via C)

Node A:  0   12   3   5
Node B: INF  0  INF INF
Node C:  3   7   0   2
Node D:  5   11   2   0
```

```
Round 8:
Node A updates distance to B from 18 to 24 (via C)
Node C updates distance to B from 21 to 19 (via D)
Node D updates distance to B from 17 to 23 (via C)

Node A:  0   24   3   5
Node B: INF  0  INF INF
Node C:  3   19   0   2
Node D:  5   23   2   0

Round 9:
Node A updates distance to B from 24 to 22 (via C)
Node C updates distance to B from 19 to 25 (via D)
Node D updates distance to B from 23 to 21 (via C)

Node A:  0   22   3   5
Node B: INF  0  INF INF
Node C:  3   25   0   2
Node D:  5   21   2   0

Round 10:
Node A updates distance to B from 22 to 28 (via C)
Node C updates distance to B from 25 to 23 (via D)
Node D updates distance to B from 21 to 27 (via C)

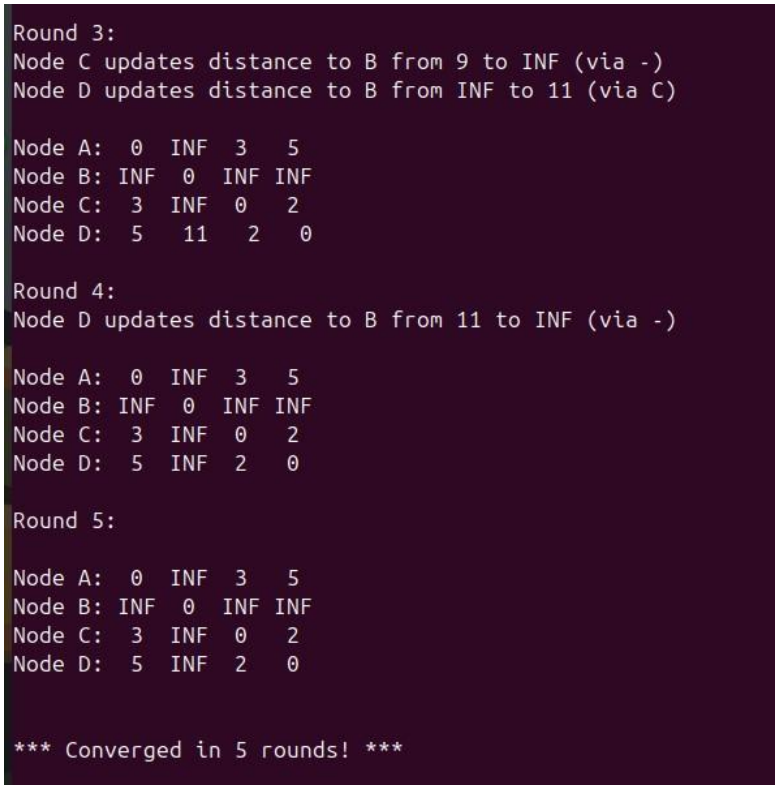
Node A:  0   28   3   5
Node B: INF  0  INF INF
Node C:  3   23   0   2
Node D:  5   27   2   0


*** Not Converged till 10 rounds! May be count to infinity problem, increase max_rounds***
```

- After the B–D link failure, Node B loses direct connectivity to several nodes, marking them as INF (unreachable).
- Neighboring nodes (A, C, D) continue to exchange distance vectors, each incorrectly believing an alternate path to B exists through others.
- As updates circulate, the cost to reach B keeps increasing gradually (6 → 12 → 14 → 20 → 28…), showing a continuous rise instead of stabilizing.

- This repetitive increase demonstrates the Count-to-Infinity problem, where nodes keep advertising outdated routes with incrementally higher costs.
- The network fails to converge even after 10 rounds, highlighting the slow and unstable nature of pure Distance Vector routing without mitigation mechanisms like Split Horizon or Poison Reverse.

b) Split Horizon



Converges in 5 Rounds

c) Poison - Reverse

```
Round 3:
Node C updates distance to B from 9 to INF (via -)
Node D updates distance to B from INF to 11 (via C)

Node A:  0  INF  3   5
Node B: INF  0  INF INF
Node C:  3  INF  0   2
Node D:  5   11   2   0

Round 4:
Node D updates distance to B from 11 to INF (via -)

Node A:  0  INF  3   5
Node B: INF  0  INF INF
Node C:  3  INF  0   2
Node D:  5  INF  2   0

Round 5:

Node A:  0  INF  3   5
Node B: INF  0  INF INF
Node C:  3  INF  0   2
Node D:  5  INF  2   0


*** Converged in 5 rounds! ***
```

Converges in 5 Rounds

Conclusion

- The assignment successfully demonstrated key concepts of Computer Networks through simulation and implementation-based experiments.
- In Problem 1, reliable data transfer was implemented using Alternating Bit Protocol (Stop-and-Wait) and Go-Back-N, analyzing retransmissions, throughput, and goodput under packet loss. GBN achieved higher efficiency due to better window utilization, while ABP ensured simpler but slower transmission.
- In Problem 3, an HTTP Proxy Server with caching was implemented, showcasing significant improvements in response time and throughput as cache hit ratio increased, validating the performance benefits of web caching.
- In Problem 4, the Distance Vector Routing Algorithm was simulated, clearly illustrating the Count-to-Infinity problem after a link failure. The Split Horizon and Poison Reverse mechanisms effectively mitigated routing loops and accelerated convergence.